CS228 : Logic for Computer Science

Module 3: Predicate Logic

Lecture 12: Syntax of Predicate Logic

Instructor: S. Akshay

IIT Bombay, India

Topics Covered till date

Module 1: Propositional Logic

1. Syntax and Semantics
   - Questions of interest: Satisfiability, Validity
2. Formal proof systems
   - Natural Deduction, Soundness and Completeness
3. Normal Forms
   - Negation, Conjunctive and Disjunctive Normal Forms (NNF, CNF, DNF)
   - Tseitin's Encoding

Module 2: The Problem of Satisfiability

1. Propositional Resolution - soundness, termination and completeness
2. The DPLL algorithm
3. Clause learning, the CDCL algorithm, and modern SAT solvers
4. A tool demo of SAT/SMT solvers in practice

Next Module: Predicate Logic

Propositional Logic was about simple statements and connectives

- e.g., If Osim is a student and Rowena is a teacher, then Osim is younger than Rowena.

Predicate logic goes beyond and talks about quantifiers and predicates

- e.g., Every student is younger than some teacher.
- What are the constituents of this sentence?
  - Being a student/teacher : these are called unary predicates.
    - e.g., S(Osim) will denote Osim is a student, T(Rowena) will denote Rowena is a teacher.
  - Being younger : these are called binary predicates.
    - e.g., Y(Osim, Rowena) will denote Osim is younger than Rowena
  - Every/some – these are quantifiers
  - Variables – to be able to say "$\forall\ x\ S(x)$" instead of writing each student out...
- e.g., $\forall x(S(x) \rightarrow \exists y(T(y) \wedge Y(x,y)))$ For all x, if x is a student, then there is some y which is a teacher such that x is younger than y.

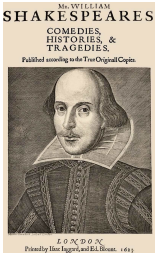
Constituents of Predicate logic

Exercise 12.1 Convert the following sentence into Predicate logic.

Not all that glitters is Gold.

- Choose predicates: $G(x)$: x glitters, $A(y)$: y is gold.
- $\neg(\forall x(G(x) \to A(x)))$
- $\exists x(G(x) \wedge \neg A(x))$

What is the difference between these two statements? Are they "equivalent"?



Can we show they are equivalent?

1. Need to define semantics, and semantical equivalence.
2. Can we also show this using proof rules? Need Natural deduction system!
3. First, what is the syntax!?

Fun fact: this proverb is a common misquote of Shakespeare's The Merchant of Venice – the original line is "All that glisters is not gold."

Topic 12.1

Syntax of Predicate Logic

Another difference from Propositional Logic

Exercise 12.2 Encode into Predicate logic.

Asif and Lavanya have the same grandfather.

- Use "variables" a for "Asif", ℓ for "Lavanya"
- Binary predicate F for "being father", i.e., $F(x, y)$ if y is father of x.

Function Symbols: Suppose we had a function symbol f(x) to mean father of x.

- Then we get an easy encoding! $f(f(a)) = f(f(\ell))$
- Function symbols are very useful to get compact encodings.
- Only applicable when no ambiguity, e.g., father of. Can't use for "brother of". (why?)
- Can take multiple or zero arguments. Functions with zero arguments are called constants.
- Thus, a and ℓ above are actually constants, not variables!

Important note: Equality $=$ is a special binary predicate symbol. Always assumed to be present!

Our Alphabet

Variable names: x, y, z

Vocabulary

- Function symbols: $f, g, h...$
  - Arity of function is number of arguments.
  - 0-ary functions are constants, we will use $a, b, c...$
- Relation (Predicate) symbols: $P, Q, R, ...$ (including $=$)
  - Hence, called Predicate logic or calculus.
  - Arity of predicate is number of arguments.

Fixed symbols

- All symbols from prop. logic: $\wedge, \vee, \neg, \rightarrow, (,)$
- New ones: $\exists, \forall$ (quantifiers)

Syntax of Predicate Logic: Terms

- Strings over the alphabet defined, but not any strings, only those with special structure!
- Two classes of syntactic objects: terms and formulas.

Terms

- every variable and constant is a term
- if f is an n-ary function and $t_1, \ldots t_n$ are terms, then so is $f(t_1, \ldots, t_n)$.

$$t ::= x \mid c \mid f(t, \ldots, t)$$

where x is over variables, c over constants and f over other function symbols.

Examples (recall Exercise 12.2)

- a (Asif), ℓ (Lavanya) are terms – they are constants.
- $f(a)$ (father of Asif), $f(f(a))$ (grandfather of Asif) are terms too!
- Careful: S(Osim) (recall the earlier example!) is *not* a term!
  - f is a function symbol: maps objects to objects, so $f(a)$ *denotes an object* – a term.
  - S is a predicate symbol: maps objects to truth values, so S(Osim) is True or False – not a term!

Syntax of Predicate Logic: Formulas

An expression that (informally) *evaluates to a truth value* is called a formula!

Formulas

- If R is an m-ary predicate, $t_1, \ldots t_m$ are terms then $R(t_1, \ldots t_m)$ is a(n atomic) formula
- Boolean connectives give formulas: $\neg\varphi$, $\varphi_1 \wedge \varphi_2$ etc.
- Quantifiers give formulas: if φ is a formula, x a variable, then so is $\exists x\varphi$, $\forall x\varphi$.

$$\varphi ::= P(t_1, \ldots t_n) \mid (\neg\varphi) \mid (\varphi \wedge \varphi) \mid (\varphi \vee \varphi) \mid (\varphi \to \varphi) \mid (\forall x\varphi) \mid (\exists x\varphi)$$

Examples

- $f(f(a)) = f(f(\ell))$ is an (atomic) formula – Asif and Lavanya have the same grandfather!
- $\forall x(S(x) \to \exists y(T(y) \wedge Y(x, y)))$ is a formula – every student is younger than some teacher.

Why "First Order"?

Called First order logic because quantification is only over individual variables...

Binding and Parse trees

Binding rules as expected!

$$\{\neg, \exists x, \forall x\} \quad > \quad \{\vee, \wedge\} \quad > \quad \{\rightarrow\}$$

- $\rightarrow$ is also right-associative: $\varphi_1 \rightarrow \varphi_2 \rightarrow \varphi_3$ means $\varphi_1 \rightarrow (\varphi_2 \rightarrow \varphi_3)$.
- Unlike $\wedge, \vee$ (where bracketing never changes the meaning), bracketing genuinely changes the truth value for $\rightarrow$!

Formulas can be represented as Parse trees as well! We omit brackets when they do not introduce ambiguity.

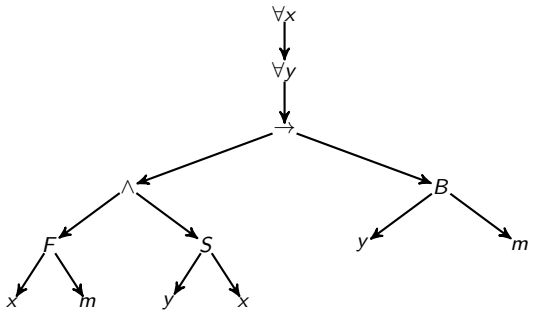
Exercise 12.3

1. Draw the Parse tree for $\forall x((P(x) \rightarrow Q(x)) \wedge S(x, y))$.
2. Encode into Predicate logic and draw its Parse tree: every son of my father is my brother.

Encoding "every son of my father is my brother"

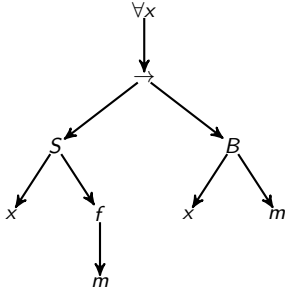
Using Predicates Only:
$F(x, y)$: x father of y; $S(x, y)$: x son of y; $B(x, y)$: x brother of y; m: "me".

$$\forall x \forall y (F(x, m) \wedge S(y, x) \rightarrow B(y, m))$$



Using a Function Symbol (reuse f from Ex 12.2!):
$S(x, y)$: x son of y; $B(x, y)$: x brother of y; m: "me".

$$\forall x (S(x, f(m)) \rightarrow B(x, m))$$



Much more compact than the predicates-only version – this is exactly why function symbols are useful!

Free variables, Bound variables and Sentences

Free variables are those that are not quantified in a formula, denoted $\mathit{free}(\varphi)$.

A recursive definition

- if φ is an atomic formula, then $\mathit{free}(\varphi) = \{x \mid x \text{ occurs in } \varphi\}$.
- if $\varphi = \neg\psi$, then $\mathit{free}(\varphi) = \mathit{free}(\psi)$.
- if $\varphi = \psi_1 \circ \psi_2$ for $\circ \in \{\wedge, \vee, \rightarrow\}$, then $\mathit{free}(\varphi) = \mathit{free}(\psi_1) \cup \mathit{free}(\psi_2)$.
- if $\varphi = Qx\ \psi$ for $Q \in \{\exists, \forall\}$, then $\mathit{free}(\varphi) = \mathit{free}(\psi) \setminus \{x\}$

If φ has free variables $\{x, y\}$, then we write $\varphi(x, y)$.
A formula with no free variables is called a sentence, e.g., $\exists x \forall y\ f(x) = y$.

Exercise 12.4

1. What is free in $\psi := ((\exists x\ P(x, y)) \wedge (\forall y\ Q(x, y)))$? Is ψ a sentence?
2. A variable is bound if it is quantified in a formula. Give a recursive definition of $\mathit{bnd}(\varphi)$.
3. Are free and bound variables complements? Why or why not?

Free and Bound Variables: Completing the Definition

Recursive definition of $bnd(\varphi)$, alongside $free(\varphi)$

- if φ is an atomic formula, then $bnd(\varphi) = \emptyset$
- if $\varphi = \neg\psi$, then $bnd(\varphi) = bnd(\psi)$
- if $\varphi = \psi_1 \circ \psi_2$ for $\circ \in \{\wedge, \vee, \rightarrow\}$, then $bnd(\varphi) = bnd(\psi_1) \cup bnd(\psi_2)$
- if $\varphi = Qx\ \psi$ for $Q \in \{\exists, \forall\}$, then $bnd(\varphi) = bnd(\psi) \cup \{x\}$

Free and Bound variables are not complements of each other!

Consider again $\psi = ((\exists x\ P(x,y)) \wedge (\forall y\ Q(x,y)))$.

$$bnd(\psi) = bnd(\exists x\ P(x,y)) \cup bnd(\forall y\ Q(x,y))$$
$$= (bnd(P(x,y)) \cup \{x\}) \cup (bnd(Q(x,y)) \cup \{y\})$$
$$= (\emptyset \cup \{x\}) \cup (\emptyset \cup \{y\}) = \{x, y\}$$

$free(\psi) = bnd(\psi) = \{x, y\}$

Substitution in FOL

Variables are placeholders; so we must be able to replace them with concrete terms: substitution.

- Suppose $x \in \mathit{free}(\varphi)$ and t is any term.
- Can we replace every free occurrence of x in φ with t?
- For example, consider $\psi = \exists y R(x, y) \vee \forall x R(z, x)$.
- If we naively try substituting $t = f(y, x)$ for x in ψ, we get $\exists y R(f(y, x), y) \vee \forall x R(z, x)$
- Problem: the y from t is now captured by $\exists y$ – it's no longer free like it was in t!

Definition

Term t is free for x in φ if no free occurrence of x in φ is in the "scope" of $\forall y$ or $\exists y$ for any var y occurring in t. e.g., $f(z, x)$ is free for x in ψ, but $f(y, x)$ is not.

Substitution

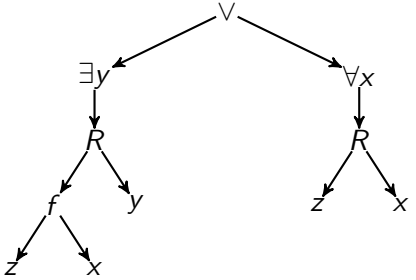
$\varphi[t/x]$ is formula obtained by replacing each free occurrence of x in φ by t, if t is free for x in φ.

For ψ defined above: $\psi[f(z, x)/x] := \exists y R(f(z, x), y) \vee \forall x R(z, x)$

Substitution in FOL: Correct vs. Captured

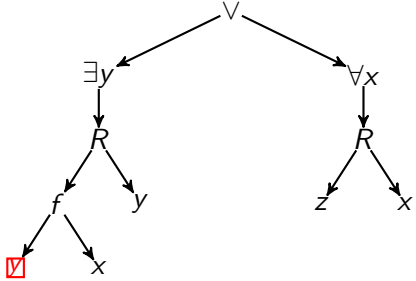
Correct: $\psi[f(z,x)/x]$

$\exists y\, R(f(z,x),y) \vee \forall x\, R(z,x)$



Wrong: $\psi[f(y,x)/x]$

$\exists y\, R(f(y,x),y) \vee \forall x\, R(z,x)$



The y inside $f(y,x)$ (boxed in red) is now captured by $\exists y$ – it's a different leaf than the term's original free y, but shares its scope!

- Fix: rename the bound y to a fresh y' first
- Rewrite ψ as $\exists y'\, R(x,y') \vee \forall x\, R(z,x)$ (same meaning, y is just a placeholder), *then* substitute.

CS228 : Logic for Computer Science

Module 3: Predicate Logic

Lecture 13: Semantics of Predicate Logic

Instructor: S. Akshay

IIT Bombay, India

Logistics

Syllabus for Mid-Semester Exam

- Module 1: Propositional Logic
- Module 2: The Problem of Satisfiability
- All lectures till and including Lecture 11

Mid-Semester Exam

- Venue: LA 201, LA 002; PwD students at KR 125
- Seating arrangement will be shared on Piazza
- 1 A4 size paper allowed as cheatsheet, but MUST be handwritten with blue ink – no photocopy, etc.

Topics Covered till date

Module 1: Propositional Logic

1. Syntax and Semantics ause
2. Formal proof systems
3. Normal Forms

Module 2: The Problem of Satisfiability

1. Propositional Resolution - soundness, termination and completeness
2. The DPLL algorithm
3. Clause learning, the CDCL algorithm, and modern SAT solvers
4. A tool demo of SAT/SMT solvers in practice

Module 3: Predicate Logic

1. Syntax of predicate logic (Lecture 12)
2. Today: semantics of predicate logic

Quick Recap: Syntax of Predicate Logic

- Terms: $t ::= x \mid c \mid f(t, \ldots, t)$
  where x is a variable, c a constant, f a function symbol
- Formulas: $\varphi ::= P(t_1, \ldots, t_n) \mid \neg\varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \varphi \to \varphi \mid \forall x\, \varphi \mid \exists x\, \varphi$
  where P is a predicate symbol
- Free/bound variables: $\mathit{free}(\varphi)$, $\mathit{bnd}(\varphi)$ defined recursively – not complements of each other!
- Substitution: $\varphi[t/x]$ replaces free occurrences of x by t – but only if t is free for x in φ (no variable of t gets captured by a quantifier in φ)!

Today: what do these formulas actually *mean*?

Topic 13.1

Semantics of Predicate Logic

Semantics of FOL

Example 13.1 $\psi_1 := \forall x \forall y (P(x,y) \rightarrow \exists z (\neg(z = x) \wedge \neg(z = y) \wedge P(x,z) \wedge P(z,y)))$

1. What does this mean? English meaning: For every x,y if $P(x,y)$ holds, then we can find z different from x, y such that $P(x,z), P(z,y)$ hold.
2. Is the statement true?
   - Case 1: Suppose,
     - variables take values from real numbers.
     - $P(x,y)$ represents $x < y$
     - Then, ψ_1 is true, i.e., real numbers are dense!
   - Case 2: Instead suppose,
     - variables take values from real numbers, but $P(x,y)$ represents $x \leq y$
     - Then, ψ_1 is false!
   - Case 3: Finally, suppose,
     - variables take values from natural numbers, with $P(x,y)$ representing $x < y$
     - Then, ψ_1 is false, since natural numbers are not dense.

Truth value of any formula φ depends on (i) underlying set from which variables take values and (ii) interpretation of constants, functions and predicates, i.e., Vocabulary!

Semantics of FOL

- Vocabulary $\mathcal{V}$. E.g., $\mathcal{V} = \{a, f, =, P\}$
- $\mathcal{V}$-formula. E.g., $\psi_2 := \exists x(P(x, f(y, a)) \wedge \neg(x = a))$, $var(\psi_2) = \{x, y\}$, $free(\psi_2) = \{y\}$.

Truth of φ depends on

1. Universe U from which variables take values, e.g., $\mathbb{N}$
2. Interpretation of $\mathcal{V}$ on U
   - Each constant to an element of U, e.g., $a \mapsto 0$
   - Each n-ary function symbol to a function from U^n to U, e.g., $f(u, v) = u + v$.
   - Each m-ary predicate symbol to a subset of U^m, e.g.,
     - Interpret for $=$: $\{(c, c) \mid c \in \mathbb{N}\}$ - fixed interpretation!
     - Interpret for P : $\{(c, d) \mid c, d \in U, c < d\}$

   This defines a $\mathcal{V}$-structure (or Model) $M = (U^M, (a^M, f^M, P^M))$, e.g., $(\mathbb{N}, (0, +, <))$
3. We also need an environment $\alpha : var(\varphi) \to U$, e.g., $\alpha(y) = 2, \alpha(x) = 1$.
   - We will also use $\alpha[x \mapsto u]$ to denote the env which maps x to u (other vars are mapped according to α).

Semantics of FOL: An Example First

- **Example 13.2** Consider again $\psi_2 := \exists x(P(x, f(y, a)) \wedge \neg(x = a))$ over Vocabulary $\mathcal{V} = \{a, f, =, P\}$
- Also, let $M = (\mathbb{N}, (0, +, <))$ be a $\mathcal{V}$-structure, and $\alpha(x) = 0, \alpha(y) = 2$ be the env.
- Does M, α satisfy ψ_2?
  - First compute $f(y, a) = y + a = 2 + 0 = 2$ (using $\alpha(y) = 2, a^M = 0$).
  - Is $P(x, f(y, a))$ true, i.e. is $x < 2$?
  - Is $\neg(x = a)$ true, i.e. is $x \neq 0$?
  - Is their conjunction true, i.e. is $x < 2$ and $x \neq 0$?
  - Finally, is there a value of x from the universe $\mathbb{N}$ that makes their conjunction true?
    - With $x = 0$ it is not satisfied.
    - But, $x = 1$ works! And that is all we need.
    - In other words, the env α, where we set $x = 1$ (ignoring the earlier value) satisfies the formula, since $(1 < 2 \wedge 1 \neq 0)$ is true!
  - Thus, we can conclude $M, \alpha \models \psi_2$!

Now, let's make this definition precise and generic!

Semantics of FOL: Formal Definition

Given $\mathcal{V}$-structure/model M and env α, we define $M, \alpha \models \varphi$ (satisfaction relation)

By structural induction! First we extend α to $\overline{\alpha} : \mathit{Terms}(\varphi) \to U^M$, by

- If t is a variable x, then $\overline{\alpha}(t) = \alpha(x)$
- If t is $f(t_1, \ldots, t_m)$, then $\overline{\alpha}(t) = f^M(\overline{\alpha}(t_1), \ldots, \overline{\alpha}(t_m))$
  In Example 13.2, we have $\overline{\alpha}(f(y, a)) = f^M(\alpha(y), a^M) = 2 + 0 = 2$

- $M, \alpha \models P(t_1, \ldots t_m)$ iff $(\overline{\alpha}(t_1), \ldots \overline{\alpha}(t_m)) \in P^M$ (atomic formulas)
- $M, \alpha \models \neg\varphi$ iff $M, \alpha \not\models \varphi$.
- $M, \alpha \models \varphi_1 \wedge \varphi_2$ iff $M, \alpha \models \varphi_1$ and $M, \alpha \models \varphi_2$.
- $M, \alpha \models \exists x\varphi$ iff there exists some $u \in U^M$ s.t., $M, \alpha[x \mapsto u] \models \varphi$.
- $M, \alpha \models \forall x\varphi$ iff for all $u \in U^M$, $M, \alpha[x \mapsto u] \models \varphi$.

We omit $\vee, \to, \Leftrightarrow$ here since they reduce to $\wedge, \neg$ in the usual way (e.g. $\varphi_1 \to \varphi_2 \equiv \neg\varphi_1 \vee \varphi_2$) – same treatment as $\wedge$ above. **Exercise 13.1** In the e.g., what if we chose (i) $\alpha(y) = 0$? (ii) $\alpha(x) = 5$? (iii) $\alpha(x) = 1$?

Satisfaction only depends on the "free" environment

Exercise 13.2

Show that if α, α' are identical on the set of free variables, then for any formula φ and any $\mathcal{V}$-structure M, $M, \alpha \models \varphi$ iff $M, \alpha' \models \varphi$.

hint: use structural induction!

- Thus, if φ has no free variables, i.e., it is a sentence, then α does not matter!

For a sentence φ, we just write $M \models \varphi$.

Recall: Truth Depends on the Structure

In Example 13.1 $\psi_1 := \forall x \forall y (P(x,y) \rightarrow \exists z (\neg(z = x) \wedge \neg(z = y) \wedge P(x,z) \wedge P(z,y)))$

- For structure $M_1 = (\mathbb{R}, <)$: $M_1 \models \psi_1$ (density!)
- For structure $M_2 = (\mathbb{R}, \leq)$: $M_2 \not\models \psi_1$
- For structure $M_3 = (\mathbb{N}, <)$: $M_3 \not\models \psi_1$

What is the vocab here?
Why is there no α?

In Example 13.2 $\psi_2 := \exists x (P(x, f(y,a)) \wedge \neg(x = a))$ over $\mathcal{V} = \{a, f, =, P\}$

- For $M_4 = (\mathbb{N}, (0, +, <))$ with $\alpha(x) = 0, \alpha(y) = 2$: $M_4, \alpha \models \psi_2$ (witness $x = 1$)
- For $M_4 = (\mathbb{N}, (0, +, <))$ with $\alpha'(x) = 0, \alpha'(y) = 0$: $M_4, \alpha' \not\models \psi_2$ (need $x < 0$, no natural number works!)

Exercise 13.3 Find a *different* structure M' (and/or environment α'') for formula ψ_2, and determine whether $M', \alpha'' \models \psi_2$.
What happens when $M' = (\mathbb{N}, (1, \times, <))$? Is there an env which makes the formula true/false?

Revisiting substitution

Exercise 13.4 Let $\psi_3(x) := \exists y\,(y > x)$ over $\mathcal{V} = \{>\}$, with $M = (\mathbb{N}, >)$.

1. What is $\mathit{free}(\psi_3)$? For any α, does $M, \alpha \models \psi_3$ hold?
2. Is $t = y$ free for x in ψ_3? If we substitute anyway, what is $\psi_3[y/x]$? What does this tell us about the substitution definition?

$\mathit{free}(\psi_3) = \{x\}$. For *any* α, is there $u \in \mathbb{N}$ with $u > \alpha(x)$? Yes, always! Numbers are unbounded above, so $M, \alpha \models \psi_3$ for every α.

Recall Substitution in FOL: $t = y$ is not free for x in ψ_3: (y occurs in t, and the free x is under the scope of $\exists y$)

- Substituting anyway, step by step:

$$\exists y\,(y > x) \quad \longrightarrow \quad \exists y\,(y > y)$$

- So $\psi_3[y/x] = \exists y\,(y > y)$
- Is *this* true? No – always false! No number is bigger than itself.

The truth value flipped. This is exactly why substitution is restricted to terms that are free for x.

Semantic entailment

Once we fix the vocab $\mathcal{V}$, we just write structure instead of $\mathcal{V}$-structure.

Let Γ be a (possibly infinite) set of formulas and let ψ be a formula in predicate logic.

- **Semantic entailment** $\Gamma \models \psi$ holds iff for all M and α, whenever $M, \alpha \models \varphi$ holds for all $\varphi \in \Gamma$, then $M, \alpha \models \psi$ holds as well.
- ψ is **satisfiable** iff there is some M and α such that $M, \alpha \models \psi$ holds.
- ψ is **valid** iff $M, \alpha \models \psi$ holds for all M and α.
- Γ is **consistent** iff there is at least one M and α such that $M, \alpha \models \varphi$ for all $\varphi \in \Gamma$.

Note: M ranges over *all* universes and *all* interpretations of $\mathcal{V}$ on them – not a fixed universe!

Exercise 13.5 Give 2 examples of valid and 2 of satisfiable (but not valid) formulas with the vocab given in Example 13.2.

Exercise 13.5 Solution

Vocab: constant a, unary function f, binary predicate P (plus always-present $=$).

Valid (true for every M and α, regardless of interpretation)

1. $a = a$ – equality has a *fixed* interpretation (always literal identity), so this holds no matter how a is interpreted.
2. $\forall x\,(P(x, f(a)) \to P(x, f(a)))$ – an instance of the propositional tautology $\varphi \to \varphi$, lifted into predicate logic; true regardless of how P, f, a are interpreted.

Satisfiable but *not* valid

1. $P(a, a)$ – true if we pick M with $P^M = U \times U$ (P always holds); false in our familiar $M_4 = (\mathbb{N}, (0, +, <))$ since $0 < 0$ is false. Satisfiable, not valid.
2. $\exists x\, P(x, a)$ – true if $U = \mathbb{Z}$, P interpreted as "$<$", $a = 0$ (witness $x = -1$); false in $M_4 = (\mathbb{N}, (0, +, <))$ since no natural number is < 0. Satisfiable, not valid.

Semantic equivalences

Definition

$\varphi \equiv \psi$ iff $\{\varphi\} \models \psi$ and $\{\psi\} \models \varphi$.

Exercise 13.6 Prove the following: Quantifier equivalences

- $\forall\, x\varphi \equiv \neg\exists\, x\neg\varphi$
- $\forall x \forall y \varphi \equiv \forall y \forall x \varphi$
- $\forall x(\varphi_1 \wedge \varphi_2) \equiv (\forall x \varphi_1) \wedge (\forall x \varphi_2)$
- $\exists x(\varphi_1 \vee \varphi_2) \equiv (\exists x \varphi_1) \vee (\exists x \varphi_2)$

Validity and Satisfiability for predicate logic

- For propositional logic, check validity just required to write down the truth table.
- Can we do something similar for predicate logic, in particular FOL?

No! Checking validity is undecidable!

Undecidable means: there is *no algorithm* which, given any formula, always correctly answers "valid" or "not valid" in finite time.

- In fact this is the case for a vocabulary with 1 constant, 2 unary functions and 1 binary predicate! (Church, 1936)
- Encoding of Post's correspondence problem.



What about satisfiability? Also undecidable!

CS228 : Logic for Computer Science

Module 3: Predicate Logic

Lecture 14: Natural Deduction for First-Order Logic

Instructor: S. Akshay

IIT Bombay, India

Welcome back after the Midsem

Module 1: Propositional Logic

1. Syntax
2. Semantics
3. Formal proof systems
4. Normal Forms

Module 2: The Problem of Satisfiability

1. Propositional Resolution
2. DPLL
3. CDCL
4. SAT/SMT tool demo

Module 3: Predicate Logic

1. Syntax of predicate logic (Lecture 12)
2. Semantics of predicate logic (Lecture 13)
3. Today: a formal proof system for predicate logic (natural deduction)

Let's warm up with a quick recap before diving into today's new material.

Predicate Logic: Alphabet and Syntax

Alphabet consists of

- Variable names: x, y, z
- Vocabulary
  - Function symbols: $f, g, h...$
  - Constants: a, b, c are just 0-ary function symbols.
  - Relation (Predicate) symbols: $P, Q, R, ...$
- Fixed symbols: $\wedge, \vee, \neg, \rightarrow, (,), \exists, \forall$

Syntax of First Order Logic (quantification over variables)

Two classes of syntactic objects: terms and formulas defined over the alphabet.

- Terms $t ::= x \mid c \mid f(t, \ldots, t)$

  where x is over variables, c over constants and f over other function symbols.

- Formulas $\varphi ::= P(t_1, \ldots t_n) \mid (\neg\varphi) \mid (\varphi \wedge \varphi) \mid (\varphi \vee \varphi) \mid (\varphi \rightarrow \varphi) \mid (\forall x\varphi) \mid (\exists x\varphi)$

  where P is over set of predicates, $t_1, \ldots t_n$ are terms and x is a variable.

Substitution in FOL

Variables are placeholders; so we must be able to replace them with concrete terms: substitution.

- Suppose $x \in \mathit{free}(\varphi)$ and t is any term.

- For example, consider $\psi = \exists y R(x, y) \vee \forall x R(z, x)$.

Definition

Term t is free for x in φ if no free occurrence of x in φ is in the "scope" of $\forall y$ or $\exists y$ for any var y occurring in t. e.g., $f(z, x)$ is free for x in ψ, but $f(y, x)$ is not.

Substitution

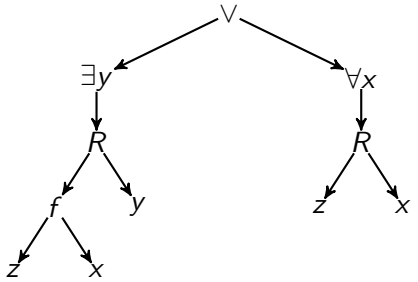
$\varphi[t/x]$ is formula obtained by replacing each free occurrence of x in φ by t, if t is free for x in φ.

For ψ defined above: $\psi[f(z, x)/x] := \exists y R(f(z, x), y) \vee \forall x R(z, x)$

Substitution in FOL: Correct vs. Captured

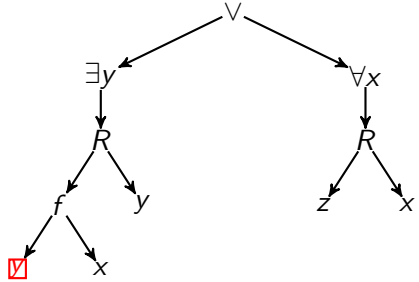
Correct: $\psi[f(z,x)/x]$

$\exists y\, R(f(z,x),y) \vee \forall x\, R(z,x)$



Wrong: $\psi[f(y,x)/x]$

$\exists y\, R(f(y,x),y) \vee \forall x\, R(z,x)$



The y inside $f(y,x)$ (boxed in red) is now captured by $\exists y$ – it's a different leaf than the term's original free y, but shares its scope!

- Fix: rename the bound y to a fresh y' first
- Rewrite ψ as $\exists y'\, R(x,y') \vee \forall x\, R(z,x)$ (same meaning, y is just a placeholder), *then* substitute.

Quick Recap: Semantics of Predicate Logic

- Vocabulary $\mathcal{V}$ + universe U + interpretation of constants/functions/predicates on $U \Rightarrow$ a $\mathcal{V}$-structure (model) M
- Environment $\alpha : var(\varphi) \to U$ (defined on all variables, but only its values on free variables affect satisfaction)
- Satisfaction relation $M, \alpha \models \varphi$ defined by structural induction on φ (atomic formulas via interpretation; $\wedge, \neg$ as usual; $\exists x\, \varphi$ iff some $u \in U$ with $M, \alpha[x \mapsto u] \models \varphi$; similarly for $\forall$)
- For a sentence (no free variables), we just write $M \models \varphi$ – the environment doesn't matter
- Semantic entailment $\Gamma \models \psi$, validity, and satisfiability are defined just as in propositional logic, with M, α in place of a truth assignment

Unlike propositional logic, both validity and satisfiability for FOL are undecidable – no algorithm can always decide them in finite time.

Today: can we nevertheless *prove* things about FOL formulas syntactically?

Topic 14.1

Natural Deduction for FOL

A Proof System for FOL

- All proof rules for Propositional Logic carry over and remain sound for FOL.
- In addition, we need new rules for:
  - Equality ($=$)
  - Quantifiers ($\forall, \exists$)

Equality Rules

- A rose is a rose!
- If two terms are equal, then we can replace one by the other in any formula.
- If $y * (w + 2) = y * w + y * 2$; then $z \geq y * (w + 2)$ implies $z \geq y * w + y * 2$ (and vice versa)

$$\frac{}{t = t} =_i \qquad \frac{t_1 = t_2 \qquad \varphi[t_1/x]}{\varphi[t_2/x]} =_e$$

- Axiom for equality, no premises
- Only on terms NOT formulas

- Of course, this is applicable ONLY if t_1, t_2 are free for x in φ.
- Eliminates $=$ by replacing all t_1 in $\varphi[t_1/x]$ by t_2.

Exercise 14.1 Show that the following sequents are valid.

1. $t_1 = t_2 \vdash t_2 = t_1$
2. $t_1 = t_2, t_2 = t_3 \vdash t_1 = t_3$

Exercise 14.1: Solution

1. $t_1 = t_2 \vdash t_2 = t_1$

1.	$t_1 = t_2$	premise
2.	$t_1 = t_1$	$=_i$
3.	$t_2 = t_1$	$=_e$ 1,2

where φ is $x = t_1$, so line 2 is $\varphi[t_1/x]$ and line 3 is $\varphi[t_2/x]$.

2. $t_1 = t_2, t_2 = t_3 \vdash t_1 = t_3$

1.	$t_1 = t_2$	premise
2.	$t_2 = t_3$	premise
3.	$t_1 = t_3$	$=_e$ 2,1

where φ is $t_1 = x$, so line 1 is $\varphi[t_2/x]$ and line 3 is $\varphi[t_3/x]$.

Proof Rules for Universal Quantification

$$\frac{\forall x \varphi}{\varphi[t/x]}\ \forall x\ e$$

$$\frac{\boxed{\begin{array}{c} x_0 \\ \vdots \\ \varphi[x_0/x] \end{array}}}{\forall x \varphi}\ \forall x\ i$$

- All students did PS1 $\Rightarrow$ so did YOU!
- If $\forall x \varphi$ holds, then we can replace x in φ by any term t.
- Imp: side condition t is free for x in φ.
  - Consider $\varphi = \exists y(x < y)$ with vocab reals with usual smaller than relation.
  - $\forall x \varphi$ means for all real n, there exists real m bigger than n, true!
  - But $\varphi[y/x] = \exists y(y < y)$, which says there must exist a no. bigger than itself, false!
  - We want only sound rules!
  - Soln: replace variable y by fresh var z

- Say I threw this tennis ball $\Rightarrow$ I can throw ANY tennis ball!
- If from a fresh var x_0, we can prove $\varphi[x_0/x]$, then $\forall x \varphi$ holds.
- Side condn: x_0 does not occur outside the box.
- In proving φ, we didn't make an assumption on x.

Examples

Exercise 14.2 Prove that the following sequents are valid.

1. $\forall x(P(x) \to Q(x)), \forall x P(x) \vdash \forall x Q(x)$
2. $P(t), \forall x(P(x) \to \neg Q(x)) \vdash \neg Q(t)$ for any term t.

Exercise 14.2: Solution

1. $\forall x(P(x) \to Q(x)), \forall x P(x) \vdash \forall x Q(x)$

1.	$\forall x(P(x) \to Q(x))$	premise
2.	$\forall x P(x)$	premise
3.	$P(x_0) \to Q(x_0)$	$\forall x$ e 1
4.	$P(x_0)$	$\forall x$ e 2
5.	$Q(x_0)$	MP 3,4
6.	$\forall x Q(x)$	$\forall x$ i 3-5

(box scoped by fresh x_0)

2. $P(t), \forall x(P(x) \to \neg Q(x)) \vdash \neg Q(t)$

1.	$P(t)$	premise
2.	$\forall x(P(x) \to \neg Q(x))$	premise
3.	$P(t) \to \neg Q(t)$	$\forall x$ e 2
4.	$\neg Q(t)$	MP 3,1

An Interesting Analogy

Intuition: $\forall$ "generalizes" $\wedge$

- Just like in $\wedge$-e, we could get F or G from $F \wedge G$, we can get $\varphi[t/x]$ for any term t (modulo side-condn!).
- Similarly $\wedge$-i had two premises, here $\forall x\ i$ has premise for any x_0.

Proof rules for Existential Quantifiers

- Can we lift this analogy and do the same for $\exists$ and $\vee$?
- For $\vee$, $\vee$-i was easy! So what should be the rule for $\exists$-i? What about $\exists$-e?

$$\frac{\varphi[t/x]}{\exists x \varphi}\ \exists x\ i$$

$$\frac{\exists x \varphi \qquad \boxed{\begin{array}{l} x_0\ \ \varphi[x_0/x] \\ \vdots \\ \chi \end{array}}}{\chi}\ \exists x\ e$$

- To use $\exists x \varphi$, you should have a proof from any possible value for x.
- The box controls scope of x_0 and scope of assumption also.

Examples

Exercise 14.3 Show that the following sequents are valid.

1. $\forall x \varphi \vdash \exists x \varphi$
2. $\forall x(P(x) \to Q(x)), \exists x P(x) \vdash \exists x Q(x)$
3. $\exists x P(x), \forall x \forall y(P(x) \to Q(y)) \vdash \forall y Q(y)$

Note: Be very careful with scope of x_0... see examples in Huth & Ryan for "false" proofs that one may derive by misusing the scope.

Many more examples in Huth & Ryan... Do try them out!

Solution for 1. $\forall x \varphi \vdash \exists x \varphi$

1.	$\forall x \varphi$	premise	
2.	φ	$\forall x\, e\, 1$	(choose $t = x$ for both steps: $\varphi[x/x] = \varphi$, so this is legal)
3.	$\exists x \varphi$	$\exists x\, i\, 2$	

Exercise 14.3.2: Correct vs. Wrong

Correct

1.	$\forall x(P(x) \to Q(x))$	premise
2.	$\exists x P(x)$	premise
3.	$P(x_0)$	assumption
4.	$P(x_0) \to Q(x_0)$	$\forall x\ e\ 1$
5.	$Q(x_0)$	MP 4,3
6.	$\exists x Q(x)$	$\exists x\ i\ 5$
7.	$\exists x Q(x)$	$\exists x\ e$ 2, 3-6

Wrong!

1.	$\forall x(P(x) \to Q(x))$	premise
2.	$\exists x P(x)$	premise
3.	$P(x_0)$	assumption
4.	$P(x_0) \to Q(x_0)$	$\forall x\ e\ 1$
5.	$Q(x_0)$	MP 4,3
6.	$Q(x_0)$	$\exists x\ e$ 2, 3-5
7.	$\exists x Q(x)$	$\exists x\ i\ 6$

The box on the right closes one line too early: line 6 ($Q(x_0)$) lets the fresh x_0 escape its own box before $\exists x\ i$ is applied outside – but χ in $\exists x\ e$ must not mention x_0!

CS228 : Logic for Computer Science

Module 3: Predicate Logic

Lecture 15: Soundness, Completeness, and Structures in FOL

Instructor: S. Akshay

IIT Bombay, India

Topics Covered till date

Module 1: Propositional Logic

1. Syntax
2. Semantics
3. Formal proof systems
4. Normal Forms

Module 2: The Problem of Satisfiability

1. Propositional Resolution
2. DPLL
3. CDCL
4. SAT/SMT tool demo

Module 3: Predicate Logic

1. Syntax of predicate logic (Lecture 12)
2. Semantics of predicate logic (Lecture 13)
3. Natural deduction for FOL (Lecture 14)
4. Today: is our proof system sound and complete, and just how powerful is FOL?

Quick Recap: Natural Deduction for FOL

- All proof rules for Propositional Logic carry over and remain sound for FOL.

$$\frac{}{t = t}\ =_i \qquad \frac{t_1 = t_2 \qquad \varphi[t_1/x]}{\varphi[t_2/x]}\ =_e$$

Equality: reflexivity axiom, and substitute equals for equals (terms only, and only where free for x).

$$\frac{\forall x \varphi}{\varphi[t/x]}\ \forall x\ e \qquad \frac{\begin{array}{|l} x_0 \\ \vdots \\ \varphi[x_0/x] \end{array}}{\forall x \varphi}\ \forall x\ i$$

Universal quantifier: instantiate to any term t (free for x); or prove $\varphi[x_0/x]$ for a fresh x_0 inside a box.

$$\frac{\varphi[t/x]}{\exists x \varphi}\ \exists x\ i \qquad \frac{\exists x \varphi \qquad \begin{array}{|ll} x_0 & \varphi[x_0/x] \\ & \vdots \\ & \chi \end{array}}{\chi}\ \exists x\ e$$

Existential quantifier: generalize $\varphi[t/x]$ to $\exists x \varphi$; or from $\exists x \varphi$ and a boxed proof of χ from fresh $x_0, \varphi[x_0/x]$, conclude χ (provided χ doesn't mention x_0).

Side conditions on freshness/scope are important – violate them and rules become unsound!

Quick Recap: (Ungraded) Pop quiz!

Recall the exercise from last lecture: Prove the following sequents using ND for FOL:

1. $\forall x \varphi \vdash \exists x \varphi$
2. $\forall x(P(x) \to Q(x)), \exists x P(x) \vdash \exists x Q(x)$
3. $\exists x P(x), \forall x \forall y (P(x) \to Q(y)) \vdash \forall y Q(y)$
   Example: A factory's policy is "if any laptop in a batch has a faulty battery, recall every laptop in the batch." We're told some laptop has a faulty battery – so every laptop gets recalled!
   ($P(x)$: x has a faulty battery; $Q(y)$: y gets recalled.)
4. "Every prime number greater than 2 is odd. There exists a prime number greater than 2. So, there exists an odd number." First formalize this in FOL, then prove it using ND!

Which rules would you use first? When do you need a fresh variable and when do you need a box?

First Question: $\forall x \varphi \vdash \exists x \varphi$

1. $\forall x \varphi \vdash \exists x \varphi$

1.	$\forall x \varphi$	premise
2.	φ	$\forall x\ e\ 1$
3.	$\exists x \varphi$	$\exists x\ i\ 2$

(choose $t = x$ for both steps: $\varphi[x/x] = \varphi$, so this is legal)

Second Question: $\forall x(P(x) \to Q(x)), \exists x P(x) \vdash \exists x Q(x)$

1.	$\forall x(P(x) \to Q(x))$	premise
2.	$\exists x P(x)$	premise
3.	x_0 $P(x_0)$	assumption
4.	$P(x_0) \to Q(x_0)$	$\forall x$ *e* 1
5.	$Q(x_0)$	MP 4,3
6.	$\exists x Q(x)$	$\exists x$ *i* 5
7.	$\exists x Q(x)$	$\exists x$ *e* 2, 3-6

Wrong!

1.	$\forall x(P(x) \to Q(x))$	premise
2.	$\exists x P(x)$	premise
3.	x_0 $P(x_0)$	assumption
4.	$P(x_0) \to Q(x_0)$	$\forall x$ *e* 1
5.	$Q(x_0)$	MP 4,3
6.	$Q(x_0)$	$\exists x$ *e* 2, 3-5
7.	$\exists x Q(x)$	$\exists x$ *i* 6

The box on the right closes one line too early: line 6 ($Q(x_0)$) lets the fresh x_0 escape its own box before $\exists x$ *i* is applied outside – but χ in $\exists x$ *e* must not mention x_0!

Third Question: $\exists x P(x), \forall x \forall y (P(x) \to Q(y)) \vdash \forall y Q(y)$

3. $\exists x P(x), \forall x \forall y (P(x) \to Q(y)) \vdash \forall y Q(y)$

1.	$\exists x P(x)$	premise
2.	$\forall x \forall y (P(x) \to Q(y))$	premise
3.	y_0	
4.	x_0 $P(x_0)$	assumption
5.	$\forall y (P(x_0) \to Q(y))$	$\forall x$ e 2
6.	$P(x_0) \to Q(y_0)$	$\forall y$ e 5
7.	$Q(y_0)$	MP 6,4
8.	$Q(y_0)$	$\exists x$ e 1, 4-7
9.	$\forall y Q(y)$	$\forall y$ i 3-8

(outer box scoped by fresh y_0; inner box scoped by fresh x_0, assuming $P(x_0)$)

Fourth Question: Formalization & Proof

Every prime number greater than 2 is odd. There exists a prime number greater than 2. So, there exists an odd number.

Formalization

Let $P(x)$: "x is a prime number greater than 2"; $Q(x)$: "x is odd."

$\forall x(P(x) \to Q(x)), \exists x P(x) \vdash \exists x Q(x)$

1.	$\forall x(P(x) \to Q(x))$	premise
2.	$\exists x P(x)$	premise
3.	x_0 $P(x_0)$	assumption
4.	$P(x_0) \to Q(x_0)$	$\forall x$ *e* 1
5.	$Q(x_0)$	MP 4,3
6.	$\exists x Q(x)$	$\exists x$ *i* 5
7.	$\exists x Q(x)$	$\exists x$ *e* 2, 3-6

Topic 15.1

Soundness, Completeness, and the Power of FOL

Soundness and completeness

Soundness: If $\varphi \vdash \psi$, then $\varphi \models \psi$

- We can only derive semantically correct proofs!
- Intuitively easy to see, again by structural induction.

Is this proof system also complete?

Completeness: If $\varphi \models \psi$, then $\varphi \vdash \psi$

- Is anything that is true, derivable?
- Yes, but we won't stop to prove it here!

So even checking whether $\vdash \varphi$ holds is undecidable – because validity itself ($\models \varphi$) is undecidable, and completeness tells us $\vdash$ and $\models$ coincide!

How powerful is First-order logic?

Even basic properties are undecidable.

Can it express everything of interest?

How can we use it if it is so powerful?

Food for thought!

Topic 15.2

Structures and Expressiveness of FOL

Quick Recap: Semantics of Predicate Logic (Lecture 13)

- Vocabulary $\mathcal{V}$: the constants, functions, and predicates available (e.g. $\mathcal{V} = \{a, f, =, P\}$).
- $\mathcal{V}$-structure (or Model) $M = (U^M, (a^M, f^M, P^M))$: a universe U^M plus an interpretation of every symbol in $\mathcal{V}$ over it (e.g. $M = (\mathbb{N}, (0, +, <))$).
- Environment α: maps free variables to elements of U^M (needed only when φ has free variables).
- Satisfaction $M, \alpha \models \varphi$: defined by structural induction on φ.

First-order Definable Structures

- Recall that, if φ is a sentence (no free variables), no environment is necessary for evaluating truth of φ
  - So given $\mathcal{V}$-structure M, we can ask if $M \models \varphi$
  - Class of $\mathcal{V}$-structures defined by φ is just $\{M \mid M \models \varphi\}$
  - Such classes of structures that are defined by an FOL sentence are called FO-definable.
- We will see three commonly occurring examples of structures:
  - Numbers (we saw this already!)
  - Graphs
  - Databases

Number systems as FO structures

Natural/reals with add, mult, lin order and constants 0,1

- $\mathfrak{N} = \{(\mathbb{N}, 0, 1, \times, +, <, =)\}$
- $\mathfrak{R} = \{(\mathbb{R}, 0, 1, \times, +, <, =)\}$

Examples of properties expressible in FOL

- Every number has a cube root
- $\mathfrak{R} \models \forall x \exists y (x = ((y \times y) \times y))$
- $\mathfrak{N} \not\models \forall x \exists y (x = ((y \times y) \times y))$
- There are infinitely many prime natural numbers. **Exercise 15.1**

Maybe we can check validity, but given the model/structure can we check if it satisfies a given formula? Model checking!

CS228 : Logic for Computer Science

Module 3: Predicate Logic

Lecture 16: Expressiveness and the Limits of FOL

Instructor: S. Akshay

IIT Bombay, India

Topics Covered till date

Module 1: Propositional Logic

1. Syntax
2. Semantics
3. Formal proof systems
4. Normal Forms

Module 2: The Problem of Satisfiability

1. Propositional Resolution
2. DPLL
3. CDCL
4. SAT/SMT tool demo

Module 3: Predicate Logic

1. Syntax of predicate logic (Lecture 12)
2. Semantics of predicate logic (Lecture 13)
3. Natural deduction for FOL (Lecture 14)
4. Soundness, completeness, and structures in FOL (Lecture 15)
5. Today: Expressiveness and the limits of FOL

Quick Recap: First-order Definable Structures

- Recall: for a sentence φ (no free variables), $\{M \mid M \models \varphi\}$ – the class of structures it defines – is called FO-definable.
- Already saw: Numbers as FO structures, $\mathfrak{N} = (\mathbb{N}, 0, 1, \times, +, <, =)$ and $\mathfrak{R} = (\mathbb{R}, 0, 1, \times, +, <, =)$.
  - E.g.for $\psi = \forall x \exists y (x = y \times y \times y)$, $\mathfrak{R} \models \psi$ (every real has a cube root), but $\mathfrak{N} \not\models \psi$; and infinitely many primes (Exercise 15.1).
- Today: Databases and Graphs as FO structures, then just how far this expressive power actually goes.

Topic 16.1

Structures and Expressiveness of FOL

Relational Databases as FO structures

A school's database has tables like `DOB` (a student's date of birth), `Class` (which class a student is in), and `Primary` (which classes are primary-school classes)

DOB	
Student	Year
s_1	2010
s_2	2018

Class	
Student	Class
s_1	c_2
s_2	c_1

Primary
c_1

Relational Database D (a $\mathcal{V}$-structure/model)

- U^D: set of (possibly differently typed) data items
- Vocab $\mathcal{V} : \{P_1, \ldots P_k, =\}$, where P_i is k_i-ary pred corr to i^{th} table in database with k_i cols.
- Interpret: For $a_1, \ldots a_{k_i} \in U^D$, $P_i(a_1, \ldots a_{k_i}) = \top$ iff $(a_1, \ldots a_{k_i})$ is a row of ith table.

Example of class of databases definable in FOL:

$\forall x \forall y \forall z \, (Record(x, y, z) \leftrightarrow (DOB(x, y) \wedge Class(x, z)))$: Record is the join of DOB and Class.

Example Query: $\varphi(x) := \exists y \exists z (DOB(x, y) \wedge After(y, \text{“2016”}) \wedge Class(x, z) \wedge Primary(z))$

Defines set of students born after 2016, and studying in a primary class.

Model checking: Does a given Database satisfy a given property?

Words as FO structures

A word structure W (a $\mathcal{V}$-structure/model)

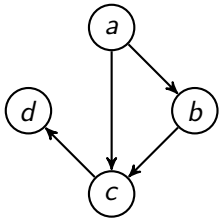
- U^W: set of all finite binary words (strings) over $\{0,1\}$, including the empty word ϵ.
- Vocab $\mathcal{V} : \{\cdot, \leq, =\}$, where $\cdot$ is a binary function symbol and $\leq$ a binary predicate symbol.
- Interpret $\cdot^W$: concatenation, e.g. $0110 \cdot^W 1110 = 01101110$; in general $a_1 \ldots a_k \cdot^W b_1 \ldots b_n = a_1 \ldots a_k b_1 \ldots b_n$.
- Interpret $\leq^W$: the prefix ordering – $s_1 \leq^W s_2$ iff $\exists s_3$ such that $s_1 \cdot^W s_3 = s_2$

E.g. 011 is a prefix of 011001 and of 011, but 010 is neither.

Pop Quiz. For each formula φ below, write its English meaning and argue whether it is true in W – i.e., does $W \models \varphi$ hold?

- **Q1.** $\exists y \, \forall x \, (y \leq x)$
- **Q2.** $\forall x \, \exists y \, (y \leq x)$
- **Q3.** $\forall x \, \forall y \, \forall z \, \big((x \leq y) \rightarrow (x \cdot z \leq y \cdot z)\big)$
- **Q4.** $\neg \exists x \, \forall y \, \big((x \leq y) \rightarrow (y \leq x)\big)$

Graphs as FO structures



A graph G (a $\mathcal{V}$-structure/model)

- U^G is a set of vertices
- Vocab $\mathcal{V} : \{E, =\}$ where E is the (binary) edge relation
- Interpret: For $a, b \in U^G$, $E^G(a, b) = \top$ iff there is a(n directed) edge from vertex a to vertex b in G.

Examples of graphs in FOL

- What does $\forall x \forall y(\neg(x = y) \rightarrow E(x, y))$ represent? (Infinite) Class of cliques!
- **Exercise 15.2** Give a FOL formula to define class of all graphs with no cycles of length 3.
  - A directed 3-cycle is $x \rightarrow y \rightarrow z \rightarrow x$ for three *distinct* vertices:

$$\exists x \exists y \exists z \left(\neg(x = y) \wedge \neg(y = z) \wedge \neg(x = z) \wedge E(x, y) \wedge E(y, z) \wedge E(z, x)\right)$$

  - So "no 3-cycles" is the negation of that:

$$\forall x \forall y \forall z \left(\neg(x = y) \wedge \neg(y = z) \wedge \neg(x = z) \wedge E(x, y) \wedge E(y, z) \rightarrow \neg E(z, x)\right)$$

Power of FOL

- FOL seems all powerful!
- Let us turn to a simpler question:
  - Given u, v in a directed graph, does there exist a finite path from u to v?
  - i.e., Give a FOL formula φ, which has u, v as free variables and E as the binary edge relation, such that
    - $G \models \varphi$ iff there is a path in G from u to v
    - So every graph G (with u, v assigned to specific vertices) satisfying φ must have a directed path from u to v
    - $u = v \vee E(u, v) \vee \exists x(E(u, x) \wedge E(x, v)) \vee \ldots$ is infinite! So not correct...

Surprise! There is no FOL formula that can express this!

Compactness of FOL

Turns out there is a very interesting property of FOL that implies this and many more results...

Compactness theorem

Let Γ be a (possibly infinite) set of FOL sentences. If all finite subsets of Γ are satisfiable/consistent, then so is Γ.

Corollary: If Γ is unsatisfiable/inconsistent, then there is a finite subset of Γ that is already unsat!
Proof of Theorem follows from completeness of FOL!

- If Γ is unsatisfiable, then $\Gamma \models \bot$ holds (why?)
- By completeness, $\Gamma \vdash \bot$ is valid, in other words there is a proof in ND for this.
- This proof is a finite set of lines, so uses only finitely many premises Δ from Γ.
- But then $\Delta \vdash \bot$ is also valid.
- By soundness, we get $\Delta \models \bot$.. thus we have a finite subset of Γ that is already unsat, as required!

Reachability is not expressible in FOL

Theorem: Reach is not expressible in FOL

- Suppose not. Then let ψ be the formula and let c, c' be consts.
- Let ψ_n be the formula expressing that there is a path of length n from c to c' **Exercise 15.3**
- Let $\Delta = \{\neg\psi_i \mid i \geq 0\} \cup \{\psi[c/u][c'/v]\}$.
- All formulas in Δ are sentences and it is unsatisfiable. (why?)
- But every finite subset of Δ is satisfiable!(why?)
- By compactness, Δ is satisfiable! Contradiction.

So how do we increase expressivity?
Quantify over sets!– higher order logic!!

Another corollary

Using compactness to show the following theorem.

Exercise 15.4

Let ψ be a FOL sentence s.t. $\forall n \geq 1$, there is a $\mathcal{V}$-structure M_n with $\geq n$ elements in its universe, such that $M_n \models \psi$, then there exists a $\mathcal{V}$-structure M with infinitely many elements in universe s.t. $M \models \psi$.

CS228 : Logic for Computer Science

Module 3: Predicate Logic

Lecture 17: Normal Forms for FOL

Instructor: S. Akshay

IIT Bombay, India

Topics Covered till date

Module 1: Propositional Logic

1. Syntax
2. Semantics
3. Formal proof systems
4. Normal Forms

Module 2: The Problem of Satisfiability

1. Propositional Resolution
2. DPLL
3. CDCL
4. SAT/SMT tool demo

Module 3: Predicate Logic

1. Syntax of predicate logic (Lecture 12)
2. Semantics of predicate logic (Lecture 13)
3. Natural deduction for FOL (Lecture 14)
4. Soundness, completeness, and structures in FOL (Lecture 15)
5. Expressiveness and the limits of FOL (Lecture 16)
6. Today: Normal forms for FOL

Pop Quiz Yet Again!

1. Let m be a constant, f a function symbol of arity 1, and S, B predicate symbols of arity 2. Which of the following are well-formed formulas? Which of them are atomic?

i.	$S(m, x)$	well-formed – atomic
ii.	$B(m, f(m))$	well-formed – atomic
iii.	$f(m)$	not a formula (a term)
iv.	$B(B(m, x), y)$	ill-formed ($B(m, x)$ isn't a term)
v.	$S(B(m), z)$	ill-formed (B wrong arity, and not a term)
vi.	$(B(x, y) \to (\exists z\, S(z, y)))$	well-formed – not atomic
vii.	$(S(x, y) \to S(y, f(f(x))))$	well-formed – not atomic
viii.	$(B(x) \to B(B(x)))$	ill-formed (wrong arity)

2. Let Q be a predicate symbol of arity 3, g a function symbol of arity 2, c a constant.
   - $\varphi_1 := \forall x \forall y\, Q(g(x, y), g(y, y), c)$ $\quad$ $\varphi_2 := \forall x \forall y\, Q(g(x, y), g(y, y), z)$
   - $\varphi_3 := \forall x \forall y \forall z\, Q(g(x, y), g(y, y), z)$ $\quad$ $\varphi_4 := \forall x \forall y \exists z\, Q(g(x, y), g(y, y), z)$

   Under $M = (\mathbb{N}, g^M, Q^M, c^M)$, with $g^M(x, y) := 0$ for all x, y; $Q^M := \{(a, b, d) \mid d = 0\}$; $c^M := 0$, what is the truth value of $\varphi_1, \varphi_2, \varphi_3, \varphi_4$?

Pop Quiz Contd.

Note first: $Q(g(x,y), g(y,y), \cdot)$ collapses to "$(\cdot = 0)$", independent of x, y.

(a) φ_1 (constant c, c^M=0): true – M alone decides it, since c^M is already fixed at 0.

(b) φ_2 (free z): cannot be determined from M alone!
z is free, so we also need an environment α

- e.g. $\alpha(z)$:=0 gives $M, \alpha \models \varphi_2$, but $\alpha'(z)$:=1 gives $M, \alpha' \not\models \varphi_2$. Same M, two different answers.

(c) φ_3 (z bound by $\forall$): false – $\forall z\,(z = 0)$ fails in $\mathbb{N}$ (e.g. $z = 1$). M alone decides it.

(d) φ_4 (z bound by $\exists$): true – $\exists z\,(z = 0)$ holds (witness $z = 0$). M alone decides it.

Moral: $\varphi_1, \varphi_3, \varphi_4$ are all sentences – M alone fixes their truth value, no environment needed.
Only φ_2 has a free variable, so it genuinely needs an environment too.

Topic 17.1

Normal Forms for FOL

Why Normal Forms, Again?

- Recall (Module 1): propositional resolution and DPLL both needed the formula in CNF first.
- FOL has the same story – but quantifiers complicate things: $\forall, \exists$ can be nested, mixed, and their variables reused.
- Goal today: strip away the quantifiers until what's left is propositional-shaped again –
  - Prenex form: push all quantifiers out front;
  - Skolemization: eliminate the $\exists$'s;
  - Clausal form: what remains is a set of clauses, just like propositional CNF.
- This is exactly the machinery we need for:
  1. FOL resolution
  2. DPLL extensions to handle FOL (SMT solvers)

Prenex Normal Form

Example: $\forall w \forall y \left(R(f(w), y) \wedge (\neg R(w, c) \vee \neg P(w))\right)$ – all quantifiers pulled out front, matrix left quantifier-free.

Prenex form

A sentence is in prenex form if it looks like $Q_1 x_1 \ldots Q_n x_n \varphi$, where each $Q_i \in \{\forall, \exists\}$ and the quantifier-free φ is called the matrix.

We do this in two steps

1. Rectify – rename apart, so no quantifier clashes.
2. Pull quantifiers to the front (via NNF)

Rectified Formulas

$\neg\big(\exists x \forall y\, R(x,y) \to \forall y \exists x\, (R(x,y) \wedge P(x))\big)$: x and y are each reused by two *different* quantifiers!

Fully bracketed, to make scopes explicit: $\neg\Big(\big(\exists x\, (\forall y\, R(x,y))\big) \to \big(\forall y\, (\exists x\, (R(x,y) \wedge P(x)))\big)\Big)$ – the two scopes never overlap, they just share names.

Rectified

A formula is rectified ("renamed apart") if no two quantifiers use the same variable, and no variable occurs both bound and free.

Renamed apart: $\neg\big(\exists x \forall y\, R(x,y) \to \forall z \exists w\, (R(w,z) \wedge P(w))\big)$

- Why OK? $\forall x.F(x) \equiv \forall y.F(y)$ whenever y is fresh (similarly for $\exists$) – renaming bound variables never changes meaning.
- Why do it? Without it, pulling a quantifier to the front can capture a variable wrongly!
  - E.g. $P(x) \wedge \forall x\, Q(x)$ – here x is *free* in $P(x)$.
  - Naively pulling $\forall x$ to the front: $\forall x\, (P(x) \wedge Q(x))$ – now x in $P(x)$ is captured, turning "some fixed x" into "for all x"!
  - Correct: rename first ($\forall x\, Q(x) \equiv \forall x'\, Q(x')$), *then* pull: $\forall x'\, (P(x) \wedge Q(x')) \equiv P(x) \wedge \forall x\, Q(x)$

Renaming Lemma

Renaming Lemma

If y does not occur in $\varphi(x)$, then $\forall x\, \varphi(x) \equiv \forall y\, \varphi(y)$ (similarly for $\exists$).

Exercise 17.1 Prove it using ND! (Recall $\forall e, \forall i$ from Lecture 14.) One direction:

1.	$\forall x\, \varphi(x)$	premise
2.	$y_0\ \varphi(y_0)$	$\forall x\, e$, 1
3.	$\forall y\, \varphi(y)$	$\forall y\, i$, 2

(box scoped by fresh y_0 – the side conditions for both rules hold precisely because y_0 is fresh. The converse runs symmetrically.)

- **Exercise 17.2** adapt this to prove the $\exists$ case.

Substitution Needs Renaming Too

Recall (PS6, Q2d): $\varphi := \exists x\,(P(y,z) \wedge \forall y\,(\neg Q(y,x) \vee P(y,z)))$.

- Substituting t for a variable is only safe when t is free for that variable in φ
- Fix, when it's not: rename the offending bound variable first (Renaming Lemma!), *then* substitute.

$\varphi[f(x)/y]$: free y sits inside $\exists x$'s scope, and $f(x)$ contains x

- Naive: $\exists x\,(P(f(x),z) \wedge \forall y(\neg Q(y,x) \vee P(y,z)))$ – the x in $f(x)$ gets captured by $\exists x$!
- Rename first: $\exists x(\ldots) \equiv \exists x'\,(P(y,z) \wedge \forall y(\neg Q(y,x') \vee P(y,z)))$
- Now substitute safely: $\exists x'\,(P(f(x),z) \wedge \forall y(\neg Q(y,x') \vee P(y,z)))$ – x in $f(x)$ stays free.

$\varphi[g(y,z)/z]$: 2nd free z sits inside $\forall y$'s scope, and $g(y,z)$ contains y

- Naive: $\exists x\,(P(y,g(y,z)) \wedge \forall y(\neg Q(y,x) \vee P(y,g(y,z))))$ – the *2nd* $g(y,z)$'s y gets captured by that very $\forall y$!
- Rename first: $\forall y(\ldots) \equiv \forall y'\,(\neg Q(y',x) \vee P(y',z))$
- Now substitute safely: $\exists x\,(P(y,g(y,z)) \wedge \forall y'(\neg Q(y',x) \vee P(y',g(y,z))))$.

More Equivalences

Recall (Lecture 13): $\varphi \equiv \psi$ iff $\{\varphi\} \models \psi$ and $\{\psi\} \models \varphi$.

- Quantifier duals: $\neg\forall x\,\varphi \equiv \exists x\,\neg\varphi$, and $\neg\exists x\,\varphi \equiv \forall x\,\neg\varphi$
- Distributivity (*same* x on both sides): $\forall x\,(\varphi \wedge \psi) \equiv (\forall x\,\varphi) \wedge (\forall x\,\psi)$, and $\exists x\,(\varphi \vee \psi) \equiv (\exists x\,\varphi) \vee (\exists x\,\psi)$

Exercise 17.3 Prove the following too – *(i) directly* (semantically), *and (ii) using ND*:

1. Pulling out of $\wedge$: $(Qx\,\varphi) \wedge \psi \equiv Qx\,(\varphi \wedge \psi)$, for $Q \in \{\forall, \exists\}$, provided $x \notin \mathit{free}(\psi)$
2. Pulling out of $\vee$: $(Qx\,\varphi) \vee \psi \equiv Qx\,(\varphi \vee \psi)$, for $Q \in \{\forall, \exists\}$, provided $x \notin \mathit{free}(\psi)$

Careful: in practice we only ever apply this to already-rectified formulas, where x doesn't occur in ψ *at all*.

Rectified Prenex Form (RPF)

A sentence is in Rectified Prenex Form if it is in prenex form ($Q_1 x_1 \dots Q_n x_n \, \varphi$) and additionally rectified: all the x_i are distinct, and none occurs free in φ.

- To convert to RPF: push negations to the atoms (NNF), then pull all quantifiers to the front.
- Note: NNF is *not* part of RPF definition – it's just a consequence of the method to pull quantifiers out safely.

Exercise 17.4 Convert to RPF

$\neg\big(\exists x \forall y \, R(x,y) \to \forall z \exists w \, (R(w,z) \wedge P(w))\big)$

$\equiv \; \exists x \forall y \, R(x,y) \; \wedge \; \neg \forall z \exists w \, (R(w,z) \wedge P(w))$ $\quad (\neg(A \to B) \equiv A \wedge \neg B)$

$\equiv \; \exists x \forall y \, R(x,y) \; \wedge \; \exists z \forall w \, \neg(R(w,z) \wedge P(w))$ $\quad$ (push $\neg$ through $\forall, \exists$: $\neg \forall z \, \varphi \equiv \exists z \, \neg \varphi$, $\neg \exists w \, \varphi \equiv \forall w \, \neg \varphi$)

$\equiv \; \exists x \forall y \, R(x,y) \; \wedge \; \exists z \forall w \, (\neg R(w,z) \vee \neg P(w))$ $\quad$ (De Morgan: $\neg(A \wedge B) \equiv \neg A \vee \neg B$)

$\equiv \; \exists z \, \forall w \, \exists x \, \forall y \, \big(R(x,y) \wedge (\neg R(w,z) \vee \neg P(w))\big)$ $\quad$ (pull quantifiers to the front one at a time – safe since rectified)

Skolemization

Example: Let $M(x)$: "x is a man", $W(x)$: "x is a woman". "For every man, there is a woman he likes": $\forall m \exists w\,(M(m) \rightarrow (W(w) \wedge \mathit{Likes}(m,w)))$. Introduce a function f picking, for each man m, one such woman: $\forall m\,(M(m) \rightarrow (W(f(m)) \wedge \mathit{Likes}(m, f(m))))$.

The Skolemization Procedure

- In a prenex sentence, replace each $\exists x$ by a fresh function of all $\forall$-variables preceding it (just a fresh constant if none precede it), substituting it for x in the matrix, then drop the $\exists$.
- Such a function/constant is called a Skolem function/Skolem constant.
- Work outermost $\exists$ first.

Continuing Exercise 17.4's RPF: $\exists z \forall w \exists x \forall y\,(R(x,y) \wedge (\neg R(w,z) \vee \neg P(w)))$.

- No $\forall$ before $\exists z$: introduce constant c: $\forall w \exists x \forall y\,(R(x,y) \wedge (\neg R(w,c) \vee \neg P(w)))$
- $\forall w$ precedes $\exists x$: introduce $f(w)$: $\forall w \forall y\,(R(f(w),y) \wedge (\neg R(w,c) \vee \neg P(w)))$

Skolem Normal Form

- Skolemization results in sentence with only universal quantifiers: Skolem Normal Form (SNF)
- SNF is equisatisfiable with the original formula, though generally *not* equivalent to it.

Clausal Form (for Resolution)

Example: continuing $\forall w \forall y \left(R(f(w), y) \wedge (\neg R(w, c) \vee \neg P(w))\right)$ from before.

- Finally, make the matrix into CNF – distribute $\vee$ over $\wedge$ wherever needed!
  - Here $R(f(w), y) \wedge (\neg R(w, c) \vee \neg P(w))$ is *already* a conjunction of clauses – nothing to do!
- Since $\forall$ distributes over $\wedge$:

$$(\forall w \forall y\, R(f(w), y)) \;\wedge\; (\forall w \forall y\, (\neg R(w, c) \vee \neg P(w)))$$

- Dropping the (now implicit) universal quantifiers, we get a set of clauses:

$$\{\, R(f(w), y),\quad \neg R(w, c) \vee \neg P(w) \,\}$$

Clausal form

A sentence is in clausal form if written as a set of clauses $C_1, \ldots, C_k$ with *all* variables implicitly universally quantified, and no quantifier symbols remain.

Careful: different clauses never share variables, even if they reuse a name – e.g. the w in $R(f(w), y)$ is not the same w as in $\neg R(w, c) \vee \neg P(w)$.

- This is exactly the input format for FOL resolution, even if we won't go into that!

CS228 : Logic for Computer Science

Module 3: Predicate Logic

Lecture 18: DPLL(T), SMT, and Moving on to Hoare Logic

Instructor: S. Akshay

IIT Bombay, India

Topics Covered till date

Module 1: Propositional Logic

1. Syntax
2. Semantics
3. Formal proof systems
4. Normal Forms

Module 2: The Problem of Satisfiability

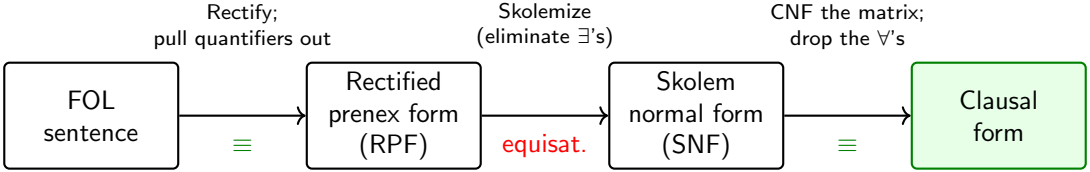
1. Propositional Resolution
2. DPLL
3. CDCL
4. SAT/SMT tool demo

Module 3: Predicate Logic

1. Syntax of predicate logic
2. Semantics of predicate logic
3. Natural deduction for FOL
4. Structures, expressiveness, and limits of FOL
5. Normal forms for FOL
6. Today: DPLL(T) and SMT solving

Quick Recap: Normal Forms So Far

- Convert FOL formulas to equisatisfiable clausal form, ready for automated reasoning.



Consider: every non-negative number has a non-negative half.

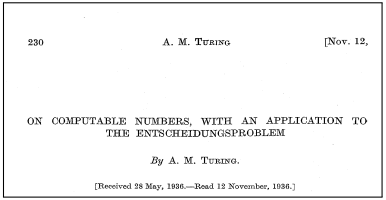
1. Write this in FOL $\varphi := \forall x \left(x \geq 0 \rightarrow \exists y \left(y \geq 0 \wedge y + y = x\right)\right)$
2. Write the RPF (one alternation) $\forall x \, \exists y \left(\neg(x \geq 0) \vee \left(y \geq 0 \wedge y + y = x\right)\right)$
3. Write the SNF (f is *new*) $\sigma := \forall x \left(\neg(x \geq 0) \vee \left(f(x) \geq 0 \wedge f(x) + f(x) = x\right)\right)$
4. Write the clausal form $\{\, \neg(x \geq 0) \vee f(x) \geq 0, \quad \neg(x \geq 0) \vee f(x) + f(x) = x \,\}$
5. What should f be in $(\mathbb{R}, 0, +, \geq, =)$? Both clauses must hold for *every* $x \in \mathbb{R}$. $f(x) := x/2$
   What if the structure is $(\mathbb{N}, 0, +, \geq, =)$? Any other f? For $x = 1$ we must have $f(1) + f(1) = 1$, but $2f(1)$ is even, so *no* f works. In fact, φ is false in $\mathbb{N}$. This does not change equisatisfiability!

Topic 18.1

The Problem of Satisfiability for FOL

The Problem of Satisfiability... for FOL

- Recall: given a propositional formula, is it satisfiable?
  - Resolution gives UNSAT proofs; DPLL/CDCL always halts with SAT/UNSAT.
- Now the same question, but for FOL: given a FOL formula, is it satisfiable?



230 A. M. TURING [Nov. 12,

ON COMPUTABLE NUMBERS, WITH AN APPLICATION TO THE ENTSCHEIDUNGSPROBLEM

By A. M. TURING.

[Received 28 May, 1936.—Read 12 November, 1936.]

Why is this harder?

- FOL validity and satisfiability are undecidable – no algorithm always halts with yes/no.
- Quantifiers range over infinite domains ($\mathbb{N}, \mathbb{R}, \ldots$), and FOL can encode computation – no finite truth table to brute-force.
- In fact, Turing introduced Turing machines while proving this for "Functional calculus K" (the old name for FOL)!

So do we give up? **No!**

- restrict to a fixed vocabulary and structure (a "theory") where satisfiability *is* decidable!

Theories: Two Examples

A theory T fixes a vocabulary $\mathcal{V}_T$ and a single intended $\mathcal{V}_T$-structure.

Example 1: LRA (Linear Real Arithmetic)

- Universe $\mathbb{R}$, $\mathcal{V}_T = \{0, 1, +, \leq, =\}$ (only linear comb, no $x \times y$), interpreted as usual.
- e.g, There are real numbers x, y such that $x \geq 5$ and $2x - y = 3$

Example 2: BV (fixed-width Bit-Vectors)

- A bit-vector of width n is simply a tuple $(b_{n-1}, \ldots, b_0) \in \{0,1\}^n$.
- Universe $\{0,1\}^n$, $\mathcal{V}_T = \{\&, |, \oplus, +, \leq_u, \ldots\}$ (bitwise ops applied per-bit, plus arithmetic mod 2^n reading the tuple as a number)
- e.g., For a 4-bit register w, doubling it *can overflow* – and become (unsigned) smaller than w itself (e.g. $w = 1100$, i.e. 12, doubled is 11000, truncated to 4 bits it's 1000, i.e. 8).

  $w + w <_u w$

  Used in hardware/crypto/systems verification.

Satisfiability Modulo Theories

The T-satisfiability problem a.k.a. Satisfiability Modulo Theories (SMT)

Given a quantifier-free $\mathcal{V}_T$-formula, is there an assignment of *values in T's universe* that makes it true?

- For both theories we considered, T-satisfiability is decidable!
  - That is, there is an algorithm that always halts with yes/no.
  - (Not every theory qualifies – e.g. general nonlinear arithmetic over the integers doesn't.)
  - LRA: It suffices to check if a conjunction of linear (in)equalities has a solution
    - How would you check if you only had equality constraints? Gaussian elimination!
    - Use "Simplex Algorithm" to check feasibility of corresponding Linear Program.
  - BV: each variable has a finite domain (2^n values) – bit-blast into an equisatisfiable propositional formula, then run usual DPLL/CDCL !

replace each n-bit variable by n Bool variables, and each op by a circuit (AND/OR/XOR gates) over those bits.

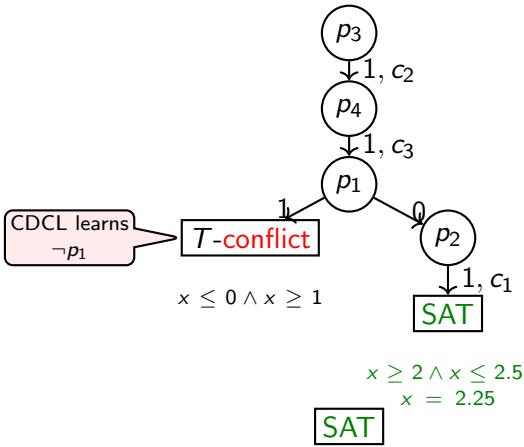
Extending DPLL to DPLL(T)

So, can we extend DPLL/CDCL to handle theories? Yes!

- Atomic $\mathcal{V}_{\mathcal{T}}$-formulas – e.g. $x \leq 5$ or $x + y = 3$ – are called theory atoms.
- Take the quantifier-free $\mathcal{V}_{\mathcal{T}}$-formula in CNF and replace each distinct theory atom by a fresh Boolean variable – the Boolean skeleton. $(x \leq 5 \vee x + y = 3) \wedge \neg(x \leq 5) \rightsquigarrow (p \vee q) \wedge \neg p$
- Run DPLL/CDCL on the skeleton to find a candidate assignment.
- Ask an underlying T-solver: is this *conjunction of theory literals* satisfiable in T's structure?
- If yes: SAT, done. If no: (learn a conflict clause and) keep searching.

DPLL(T): A Worked Example (LRA)

- Input: A quantifier-free LRA-formula in CNF $\varphi' := (x \leq 0 \vee x \geq 2) \wedge (x \geq 1) \wedge (x \leq 2.5)$.
- Theory atoms $\rightarrow$ Bool variables: $p_1{:=}(x \leq 0)$, $p_2{:=}(x \geq 2)$, $p_3{:=}(x \geq 1)$, $p_4{:=}(x \leq 2.5)$.
- Boolean skeleton: $c_1 \wedge c_2 \wedge c_3$, where $c_1{:=}(p_1 \vee p_2)$, $c_2{:=}p_3$, $c_3{:=}p_4$.



- p_1 was the only real decision – both values lead to a theory conflict, so φ is UNSAT.
- What if $c_3 := (x \leq 2.5)$ instead? Only the p_2 branch changes
- T-solver check: $\{x \geq 1, x \leq 2.5, x > 0, x \geq 2\}$ – SAT! witness $x = 2.25$.
- So φ' is satisfiable – the $p_1{=}0, p_2{=}1$ branch succeeded.

Adding Quantifiers

- In the modified, satisfiable version: $\varphi' := (x \leq 0 \vee x \geq 2) \wedge (x \geq 1) \wedge (x \leq 2.5)$.
- $\exists x\, \varphi'(x)$: true – the witness $x = 2.25$ comes for free: it is the model the T-solver already found!
- $\forall x\, \varphi'(x)$ is true iff $\neg\varphi'(x)$ is UNSAT – again just DPLL(T)!
  - $\neg\varphi'(x) \equiv (x > 0 \wedge x < 2) \vee x < 1 \vee x > 2.5$
  - It is SAT, e.g. x=0 $(0 < 1)$.
  - So $\forall x\, \varphi'(x)$ is false.
- Bottomline: one block of $\exists$ or of $\forall$ over a quantifier-free formula is just SAT/UNSAT. The trouble starts when quantifiers alternate!

Quantifier Alternation is Hard

$\varphi'(x) := (x \leq 0 \vee x \geq 2) \wedge (x \geq 1) \wedge (x \leq 2.5)$.

Pure existential

- $\exists x\, \varphi'(x)$: true, witness $x = 2.25$ – just the SAT problem DPLL(T) already solved!

Now alternate: $\forall y\, \exists x$

- Let $\psi(x, y) := (x \leq y \vee x \geq y{+}2) \wedge (x \geq y{+}1) \wedge (x \leq y{+}2.5)$ -same shape as φ', shifted by y.
- $\forall y\, \exists x\, \psi(x, y)$: for *every* y, need a witness x – but now it can (must!) depend on y. A single fixed x won't work for all y at once.
- Here $x := y + 2.25$ works for *every* y (the old witness, shifted) – that dependency is exactly a Skolem function!
- In general, *finding* that function (or enough instantiations to pin it down) is the hard part – that's where the real search/work in quantifier instantiation lives.

Summary

- Satisfiability for FOL is undecidable – so fix a theory T (LRA, BV, ...) where it is decidable: SMT.
- DPLL(T) = CDCL on the Boolean skeleton + a T-solver for conjunctions of theory literals.
- *Quantifier-free* is somewhat easy, depending on the theory; so is even one block of quantifiers (just SAT/UNSAT in T) – but alternating quantifiers are hard!

That wraps up our module on first-order logic

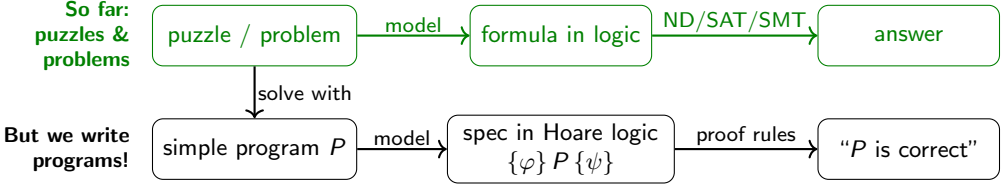
- Payoff: SMT solvers such as Z3 put much of the power of FOL to work – with decision procedures inside the theories (and heuristics beyond).
- Optional Z3 demo soon!

Topic 18.2

Introduction to Program Verification

Next topic alert!

An introduction to Program Verification



- Can we use logic to reason about programs, too?
- How? A logic of programs, Hoare logic, built on top of FOL!
- In this course: for *simple* imperative programs, we'll see how to *state* properties in logic (pre/postconditions) and *prove* them using proof rules.
- Real program verifiers (e.g., Dafny) largely automate the proof step using SMT solvers!

Remember Ariane 5? (Lecture 0)



June 4, 1996 – Ariane 5, Flight 501

- Ariane 4 code, reused in Ariane 5, overflowed – the rocket was lost within a minute of lift-off.
- On Ariane 4 that value never got so large: the code was correct under Ariane 4's assumptions, but not under Ariane 5's.
- A violated precondition!

What does "correct" mean?

"Correct" only makes sense relative to an assumed precondition and a promised postcondition

How do we formalize this in logic?

CS228 : Logic for Computer Science

Module 4: Hoare Logic and Program Verification

Lecture 19: Introduction to Hoare Logic

Instructor: S. Akshay

IIT Bombay, India

Logistics

Quiz 2

- **Friday, Oct 23rd, 6pm**
- Syllabus:
  - Module 3 (Predicate Logic/FOL): all of Lectures 12–18
  - Module 4 (Hoare logic): from Lecture 19 onwards, covered till Thursday, Oct 15th

SMT solver demo

- Postponed to next week.

Topics Covered till date

Module 1: Propositional Logic

1. Syntax
2. Semantics
3. Formal proof systems
4. Normal Forms

Module 2: The Problem of Satisfiability

1. Propositional Resolution
2. DPLL
3. CDCL
4. SAT/SMT tool demo

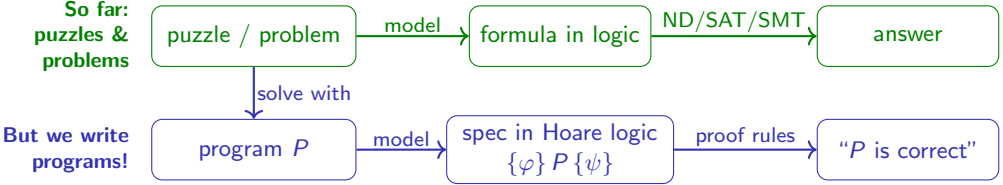
Module 3: Predicate Logic

1. Syntax of predicate logic
2. Semantics of predicate logic
3. Natural deduction for FOL
4. Structures, expressiveness, and limits of FOL
5. Normal forms for FOL
6. DPLL(T) and SMT solving

Module 4: Hoare Logic and Program Verification

Today: Introduction to Hoare logic

Recap: Hoare Logic and Program Verification



- A logic to reason about programs
- A logic of programs, Hoare logic, built on top of FOL!
- In this module: for *simple* imperative programs, we'll see how to *state* properties in logic (pre/postconditions) and *prove* them using proof rules.
- Real program verifiers (e.g., Dafny) largely automate the proof step using SMT solvers!

Consider a simple program

What does this program do? Take x as input and y as output.

	$y := 1;$
	$z := 0;$
Fac$_1$	**while** $(z \neq x)\{$
inp x	$\quad z := z + 1;$
out y	$\quad y := y * z;$
	$\}$

Trace for $x = 3$:

after	z	y	$z \neq x$?
start	0	1	yes
iter. 1	1	1	yes
iter. 2	2	2	yes
iter. 3	3	6	no (stop)

Let's call this **Fac**$_1$.

- **Exercise 19.1** What is y when the loop stops for input (i) $x = 3$ and (ii) $x = 0$? $y = 6 = 3!, y = 1 = 0!$
- The program is meant to compute the factorial of x and to store that in y.
- But does it do that? How do we show that it really computes $x!$ in y?
- What happens if we start with $x = -2$? It never stops: $z = 1, 2, 3, \ldots$ never equals -2!
- In fact, if $x \geq 0$, then **Fac**$_1$ computes $x!$ and stores it in y. Our goal is to prove this!

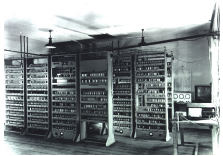
Proving programs correct

1. If $\mathbf{Fac}_1$ is run with $x \geq 0$, it terminates with $y = x!$.
2. Whenever $\mathbf{Fac}_1$ terminates, we have $y = x!$.

Our goal is to prove BOTH these statements!

- Proof goals mix a program with facts about its input/output values,
- so we treat both as mathematical/logical objects.

We explore a paradigm that can do this: Hoare Logic



EDSAC (1948)

In 1949, at a Cambridge conference, Turing asked *"How can one check a routine in the sense of making sure that it is right?"* His example was a factorial program, with assertions for correctness and a quantity that decreases and vanishes for termination – exactly our two statements!



Alan Turing

To reason about programs, we must first write them formally

- To prove things about programs, programs must be mathematical objects, just as formulas were.
- So, as for logic, we need
  - Syntax: precisely which strings are programs? (a grammar)
  - Semantics: precisely what does a program do? (how it changes the state)
- Where have we done this before? Propositional logic (Lectures 1–2) and predicate logic (Lectures 12–13)!

Topic 19.1

A Simple Programming Language

A simple (imperative) programming language

Integer expressions

$$E ::= n \mid x \mid (-E) \mid (E + E) \mid (E - E) \mid (E * E)$$

Boolean expressions

$$B ::= \top \mid \bot \mid (\neg B) \mid (B \wedge B) \mid (E \leq E)$$

Commands, i.e., programs

$$C ::= (x := E) \mid C; C \mid \textbf{if } B \ \{C\} \textbf{ else } \{C\} \mid \textbf{while } B \ \{C\}$$

where { and } mark the start and end of blocks of code

Syntax and semantics of a simple program

Fix variables $x, y, z, \ldots$, numerals/integers $n \in \{0, 1, 2, \ldots, -1, -2, \ldots\}$

$$E ::= n \mid x \mid (-E) \mid (E + E) \mid (E - E) \mid (E * E)$$
$$B ::= \top \mid \bot \mid (\neg B) \mid (B \wedge B) \mid (E \leq E)$$
$$C ::= (x := E) \mid C; C \mid \textbf{if } B \ \{C\} \textbf{ else } \{C\} \mid \textbf{while } B \ \{C\}$$

The state of the program is determined by the current value of all its variables. As the program executes, its state changes.

- $(x := E)$ is the usual assignment statement.
- $C; C$ is sequential composition.
- **if** $B \ \{C_1\}$ **else** $\{C_2\}$ evaluates boolean expression B in current state and if that result is true, then C_1 is executed, else C_2 is executed.
- **while** $B \ \{C\}$ evaluates B at current state. If B evaluates false the command terminates. Else, C is executed. When C terminates we re-evaluate B with updated state and continue. Thus, we repeatedly evaluate C as long as B is true.

The simple programming language

Exercise 19.4

1. Write a program in the above syntax to
   1.1 compute 2^n from n.
   1.2 find greatest y whose square is less than x
2. What features of a common imperative (i.e., state based) programming language like C, java, are not captured? Give 4 examples. Is it easy to modify the language to have these features?

Topic 19.2

Specifying Programs: Hoare Triples

Hoare Logic, sometimes called Floyd–Hoare Logic

Main idea: Use logic to capture what must be true before and after a command!

In general, a **Hoare triple** $\{\varphi\}\, P\, \{\psi\}$ is a formula of Hoare logic:

- φ and ψ are formulas in a base logic, e.g., predicate logic, Presburger arithmetic, etc. In our case, we will use first order logic over integers.
  E.g., for $\mathbf{Fac}_1$: $\{x \geq 0\}\ \mathbf{Fac}_1\ \{y = x!\}$
- P is an imperative program in our language
- φ is called the precondition of P and ψ the postcondition.



Robert Floyd



Tony Hoare

Floyd 1967, Hoare 1969

Turing Awards: 1978, 1980

States and formulas

States and Formulas

- A state of the program is an environment, i.e., a function α that assigns to each variable x an integer $\alpha(x)$ (as in Lecture 13).
- Assertions φ, ψ are formulas of first order logic over the vocabulary $\mathcal{V}$ with the numerals as constants, function symbols $-$ (unary) and $+, *, -$ (binary), and predicate symbols $\leq$ and $=$.
- Let $M_{\mathbb{Z}} = (\mathbb{Z}, (\ldots, -1, 0, 1, \ldots, -, +, *, \leq))$ be the $\mathcal{V}$-structure in which every symbol has its usual meaning. We write $\alpha \models \varphi$ for $M_{\mathbb{Z}}, \alpha \models \varphi$.

Exercise 19.5

Let α be a state where $\alpha(x) = -2, \alpha(y) = 5, \alpha(z) = -1$. Which of the following hold?

(a) $\alpha \models \neg(x + y < z)$

(b) $\alpha \models \forall u\,(y < u \rightarrow y * z < u * z)$

(a) holds: $x + y = 3$ is not less than $z = -1$. (b) fails: for $u = 6$, $y * z = -5 < -6 = u * z$ is false.

Examples of Hoare Triples

Some examples of correct Hoare triples

- $\{(n \geq 0) \wedge (n * n > 20)\}\ \mathrm{m} := \mathrm{n} + 1; \mathrm{m} := \mathrm{m} * \mathrm{m}\ \{\neg(m = 50)\}$ quantifier-free first order logic over integers.
- $\{\exists z\,(z + z = x)\}\ \mathrm{y} := \mathrm{x} + 2\ \{\exists z\,(z + z = y)\}$ general first order logic over integers: "if x is even, so is y".

An incorrect Hoare triple

$\{n \geq 0\}\ \mathrm{m} := \mathrm{n} + 1; \mathrm{m} := \mathrm{m} * \mathrm{m}\ \{m \geq 4\}$ – fails for $n = 0$: then $m = 1$.

Exercise 19.6

Give 3 "correct" and 3 "incorrect" Hoare triples.

How do we formalize the notion of correctness?

Correctness and semantics of Hoare Triples

Partial correctness

A triple $\{\varphi\}P\{\psi\}$ is satisfied under partial correctness, denoted $\models_{par} \{\varphi\}P\{\psi\}$ if, for all states that satisfy φ, the state resulting from P's execution satisfies the postcondition ψ, provided P actually terminates.

Total correctness

A triple $\{\varphi\}P\{\psi\}$ is satisfied under total correctness, denoted $\models_{tot} \{\varphi\}P\{\psi\}$ if, for all states that satisfy φ, P when executed is guaranteed to terminate and the resulting state satisfies the postcondition ψ.

Examples: Recall the program $\mathbf{Fac}_1$ from earlier,

- $\{\top\}\ \mathbf{Fac}_1\ \{y = x!\}$ satisfies partial correctness, but not total correctness.
- $\{x \geq 0\}\ \mathbf{Fac}_1\ \{y = x!\}$ satisfies partial AND total correctness.

Examples

Fac$_1$ *inp* x *out* y	$y := 1;$ $z := 0;$ **while** $(z \neq x)\{$ $\quad z := z + 1;$ $\quad y := y * z;$ $\}$

Unk$_1$ *inp* x *out* y	$a := x + 1;$ **if** $(a - 1 = 0)\{$ $\quad y := 1;$ $\}$ **else** $\{$ $\quad y := a;$ $\}$

- $\{\top\}$ **Fac$_1$** $\{y = x!\}$ satisfies partial correctness, but NOT total correctness.
- $\{x \geq 0\}$ **Fac$_1$** $\{y = x!\}$ satisfies partial AND total correctness.

Exercise 19.7

1. What does Unk_1 do? Take x as input and y as output.
2. Write a Hoare triple that captures the spec. Does it hold under partial/total correctness?

Program variables and logical variables

Fac$_1$ *inp* x *out* y	$y := 1;$ $z := 0;$ **while** $(z \neq x)\{$ $\quad z := z + 1;$ $\quad y := y * z;$ $\}$

Fac$_2$ *inp* x *out* y	$y := 1;$ **while** $(x \neq 0)\{$ $\quad y := y * x;$ $\quad x := x - 1;$ $\}$

- **Fac**$_2$ is another program that computes the factorial.
- $\{x \geq 0\}$ **Fac**$_1$ $\{y = x!\}$ satisfies partial and total correctness.
- Qn: What about $\{x \geq 0\}$ **Fac**$_2$ $\{y = x!\}$?
- How do we correct this? What is the correct specification? Need to remember initial value!
- we obtain $\{x = x_0 \wedge x \geq 0\}$ **Fac**$_2$ $\{y = x_0!\}$ x_0 is called a logical variable.

CS228 : Logic for Computer Science

Module 4: Hoare Logic and Program Verification

Lecture 20: Proof Rules for Hoare Logic

Instructor: S. Akshay

IIT Bombay, India

Logistics

Recall: Quiz 2

- **Friday, Oct 23rd, 6pm**
- Syllabus:
  - Module 3 (Predicate Logic/FOL): all of Lectures 12–18
  - Module 4 (Hoare logic): from Lecture 19 up to and including Thursday's lecture (Oct 15th)

Optional SMT solver demo

- **Wednesday, Oct 14th, 3:30pm**
- Venue: CC 103

Topics Covered till date

Module 1: Propositional Logic

1. Syntax
2. Semantics
3. Formal proof systems
4. Normal Forms

Module 2: The Problem of Satisfiability

1. Propositional Resolution
2. DPLL
3. CDCL
4. SAT tool demo

Module 3: Predicate Logic

1. Syntax of predicate logic
2. Semantics of predicate logic
3. Natural deduction for FOL
4. Structures, power and limits of FOL
5. Normal forms, DPLL(T) and SMT

Module 4: Hoare Logic and Program Verification

1. Introduction to Hoare logic
2. Today: Proof rules for Hoare logic

Recap: Hoare triples and correctness

A Hoare triple $\{\varphi\}\ P\ \{\psi\}$: precondition, program, postcondition

- Partial correctness ($\models_{par}$): *if* P terminates when started in a state satisfying φ, then ψ holds in the final state.
- Total correctness ($\models_{tot}$): started in any state satisfying φ, P *does terminate*, and ψ holds in the final state.

Fac$_1$ *inp* x *out* y	$y := 1;$ $z := 0;$ **while** $(z \neq x)\{$ $\quad z := z + 1;$ $\quad y := y * z;$ $\}$

Qn: Recall **Fac**$_1$. Which of these hold?

	partial	total
$\{\top\}$ **Fac**$_1$ $\{y = x!\}$	✓	×
$\{x \geq 0\}$ **Fac**$_1$ $\{y = x!\}$	✓	✓

- The two statements from Lecture 19: statement 1 (terminates with $y = x!$ if $x \geq 0$) is the *total* entry for $x \geq 0$; statement 2 (whenever it terminates, $y = x!$) is the *partial* entry for $\top$.

total correctness = partial correctness + termination

Recap: Partial correctness and non-termination

Exercise 20.1 (a): $\mathbf{Fac}_1$ never stops when $x = -2$.

- Why does $\{\top\}\ \mathbf{Fac}_1\ \{y = x!\}$ still hold under partial correctness?
- Vacuous! No terminating run starts with $x < 0$, so no run can violate $y = x!$.

Exercise 20.1 (b): Let P be **while** $\top$ $\{x := 0\}$.

- Does $\{\top\}\ P\ \{\bot\}$ hold under partial correctness? Under total correctness?
  - Note: the $\top$ in the guard is part of the *program* (a Boolean expression, like **while**); the plain $\top$ and $\bot$ in the braces are *assertions*, i.e., formulas of the logic (you can think of them as abbreviations of, e.g., $0 \leq 0$ and $\neg(0 \leq 0)$).
- Partial: yes (it never terminates, so again vacuous). Total: no.

A partially correct program may do nothing useful: partial correctness says nothing about termination!

Program variables and logical variables

Fac$_1$ *inp* x *out* y	$y := 1;$ $z := 0;$ **while** $(z \neq x)\{$ $\quad z := z + 1;$ $\quad y := y * z;$ $\}$

Fac$_2$ *inp* x *out* y	$y := 1;$ **while** $(x \neq 0)\{$ $\quad y := y * x;$ $\quad x := x - 1;$ $\}$

- **Fac**$_2$ is another program that computes the factorial.
- $\{x \geq 0\}$ **Fac**$_1$ $\{y = x!\}$ satisfies partial and total correctness.
- Qn: What about $\{x \geq 0\}$ **Fac**$_2$ $\{y = x!\}$?
- How do we correct this? What is the correct specification? Need to remember initial value!
- We obtain $\{x = x_0 \wedge x \geq 0\}$ **Fac**$_2$ $\{y = x_0!\}$ x_0 is called a logical variable.

For a Hoare triple $\{\varphi\}P\{\psi\}$, its logical variables are those that are free in φ or ψ and do not occur in P.

Topic 20.1

Proving Programs: Proof Rules

A proof calculus for program correctness

Rule1: Sequential Composition

$$\frac{\{\varphi\}\ C_1\ \{\eta\} \qquad \{\eta\}\ C_2\ \{\psi\}}{\{\varphi\}\ C_1; C_2\ \{\psi\}}\ \text{Seq. Composition}$$

- Easier to read the rules bottom-up!
- To show that ψ holds on running $C_1; C_2$ starting from φ, we have to find η and show that η holds after running C_1 from φ and from η running C_2 results in ψ holding.
- η is called a midcondition.

Example:

$$\frac{\{y+z>4\}\ y := y+z-1\ \{y>3\} \qquad \{y>3\}\ x := y+4\ \{x>7\}}{\{y+z>4\}\ y := y+z-1; x := y+4\ \{x>7\}}\ \text{Seq. Composition}$$

Hoare Logic: Proof rule for Assignment

Rule 2: Assignment

$$\frac{}{\{\psi[E/x]\}\ x := E\ \{\psi\}}\ \text{Assignment}$$

- This rule is an axiom.
- $\psi[E/x]$ refers to replacing every *free* occurrence of x in ψ by E.
- Example: $\{x + 1 + 5 = y\}\ x := x + 1\ \{x + 5 = y\}$
- Why are we doing this "backward"? Would forward work?
- No! $\{\varphi\}\ x := E\ \{\varphi[E/x]\}$ does not hold in general. **Exercise 20.2**: Show this by giving a counter-example.

Exercise 20.3: Let P be $x := 2$. Are these true? (i) $\{2 = 2\}\ P\ \{x = 2\}$, (ii) $\{x = 2\}\ P\ \{2 = 2\}$, (iii) $\{x = 3\}\ P\ \{x = 3\}$, (iv) $\{2 = 4\}\ P\ \{x = 4\}$. Sol: Only (i), (ii), (iv) are true.

Hoare Logic: Proof rule for If-then-Else

Rule 3: If-then-Else

$$\frac{\{\varphi \wedge B\}\ C_1\ \{\psi\} \qquad \{\varphi \wedge \neg B\}\ C_2\ \{\psi\}}{\{\varphi\}\ \textbf{if}\ B\ \{C_1\}\ \textbf{else}\ \{C_2\}\ \{\psi\}}\ \text{If-then-Else}$$

- In the *then*-branch we know B held; in the *else*-branch we know B failed. We use this information in the sub-proofs!

Example

$$\frac{\{x > 4 \wedge z > 0\}\ x := x + z\ \{x > 3\} \qquad \{x > 4 \wedge \neg(z > 0)\}\ x := x - 1\ \{x > 3\}}{\{x > 4\}\ \textbf{if}\ (z > 0)\ \{x := x + z\}\ \textbf{else}\ \{x := x - 1\}\ \{x > 3\}}\ \text{If-then-Else}$$

(Each premise needs to be proved (using other rules).)

Hoare Logic: Strengthening Preconditions, Weakening Postconditions

Rule 4: Strengthen the precondition, weaken the postcondition

$$\frac{\vdash_{AR} \varphi \to \varphi' \qquad \{\varphi'\}\ C\ \{\psi'\} \qquad \vdash_{AR} \psi' \to \psi}{\{\varphi\}\ C\ \{\psi\}}\ \text{Implied}$$

- The rule links Hoare logic to the base logic: $\vdash_{AR} \varphi \to \varphi'$ means the implication is provable in natural deduction for first order logic, taking facts about arithmetic (e.g., $\forall x\,(x = x + 0)$) as premises.
- Strengthen the precondition = assume more than needed; weaken the postcondition = promise less.
- Note that this works regardless of C!

Example

$$\frac{\vdash_{AR} x > 9 \to x > 7 \qquad \{x > 7\}\ x := x - 4\ \{x > 3\} \qquad \vdash_{AR} x > 3 \to x > 2}{\{x > 9\}\ x := x - 4\ \{x > 2\}}\ \text{Implied}$$

Hoare Logic: Proof rule for partial correctness of While Loop

Rule 5: partial-while

$$\frac{\{\eta \wedge B\}\ C\ \{\eta\}}{\{\eta\}\ \textbf{while}\ B\ \{C\}\ \{\eta \wedge \neg B\}}\ \text{Partial-while}$$

- η is a loop-invariant: it holds before the loop and is preserved by every iteration (that is what the premise says).
- Partial correctness!
  - If loop does not terminate, Hoare triple is vacuously satisfied.
  - If it does terminate, then $\eta \wedge \neg B$ must be satisfied at termination.
- Example

$$\frac{\{y = x + z \wedge (z \neq 0)\}\ x := x + 1; z := z - 1\ \{y = x + z\}}{\{y = x + z\}\ \textbf{while}\ (z \neq 0)\ \{x := x + 1; z := z - 1\}\ \{y = x + z \wedge (z = 0)\}}$$

Topic 20.2

Using the Proof Rules: Proof Outlines and Loop Invariants

A proof calculus for program correctness

$$\frac{\{\varphi\}\ C_1\ \{\eta\} \qquad \{\eta\}\ C_2\ \{\psi\}}{\{\varphi\}\ C_1; C_2\ \{\psi\}}\ \text{Seq. Comp.}$$

$$\frac{\{\varphi \wedge B\}\ C_1\ \{\psi\} \qquad \{\varphi \wedge \neg B\}\ C_2\ \{\psi\}}{\{\varphi\}\ \textbf{if}\ B\ \{C_1\}\ \textbf{else}\ \{C_2\}\ \{\psi\}}\ \text{If-then-Else}$$

$$\frac{}{\{\psi[E/x]\}\ x := E\ \{\psi\}}\ \text{Assignment}$$

$$\frac{\vdash_{AR} \varphi \to \varphi' \qquad \{\varphi'\}\ C\ \{\psi'\} \qquad \vdash_{AR} \psi' \to \psi}{\{\varphi\}\ C\ \{\psi\}}\ \text{Implied}$$

$$\frac{\{\eta \wedge B\}\ C\ \{\eta\}}{\{\eta\}\ \textbf{while}\ B\ \{C\}\ \{\eta \wedge \neg B\}}\ \text{Partial-while}$$

We say sequent $\vdash_{par} \{\varphi\}\ P\ \{\psi\}$ is valid if ψ can be derived from φ by applying these rules!

This proof system is sound so we can use this to infer partial correctness, i.e., $\models_{par}$.

Using the proof rules in practice!

Writing trees is unwieldy (and difficult in latex!). So we use a different convention...

Example 1: Show that $\vdash_{par} \{y < 5\}\ x := y + 1\ \{x < 6\}$

$\{y < 5\}$

$\{y + 1 < 6\}$ Implied

$x := y + 1$

$\{x < 6\}$ Assignment

1. Start by writing down the program
2. Write down the pre- and post-conditions before and after
3. Work backwards using rule corresponding to prog. statement to get a mid condition
4. At any midcondition, you can go further up by using other rules

Another example

Consider program P, which is $z := x; z := z + y; u := z$. **Exercise 20.4**. Write a Hoare triple!

Example 2: Show that $\vdash_{par} \{\top\}\ P\ \{u = x + y\}$

$$
\begin{array}{ll}
\{\top\} & \\
\{x + y = x + y\} & \text{Implied} \\
z := x; & \\
\{x + y = z + y\} & \text{Assignment} \\
z := z + y; & \\
\{x + y = z\} & \text{Assignment} \\
u := z; & \\
\{u = x + y\} & \text{Assignment}
\end{array}
$$

- Though you constructed the proof from bottom to up, you can read it top to bottom!
- Implied rule always refers to immediate preceding line.
- Weakest pre-condition

Using the If-then-Else rule

$$\frac{\{\varphi \wedge B\}\ C_1\ \{\psi\} \qquad \{\varphi \wedge \neg B\}\ C_2\ \{\psi\}}{\{\varphi\}\ \textbf{if}\ B\ \{C_1\}\ \textbf{else}\ \{C_2\}\ \{\psi\}}\ \text{If-then-Else}$$

$$\frac{\{\varphi_1\}\ C_1\ \{\psi\} \qquad \{\varphi_2\}\ C_2\ \{\psi\}}{\{(B \to \varphi_1) \wedge (\neg B \to \varphi_2)\}\ \textbf{if}\ B\ \{C_1\}\ \textbf{else}\ \{C_2\}\ \{\psi\}}\ \text{If-then-Else}$$

We will use a slight variant of the rule in practice...

Exercise 20.5

- Show that any instance of the modified rule in a proof can be replaced by an instance of the original rule and instances of rule Implied.
- Is the converse true?

CS228 : Logic for Computer Science

Module 4: Hoare Logic and Program Verification

Lecture 21: Proving Loops Correct: Invariants and Variants

Instructor: S. Akshay

IIT Bombay, India

Logistics

Recall: Quiz 2

- **Friday, Oct 23rd, 6pm**
- Venue: CC 101/103/105 (*to be confirmed, will be announced on Piazza*)
- Syllabus:
  - Module 3 (Predicate Logic/FOL): all of Lectures 12–18
  - Module 4 (Hoare logic): from Lecture 19 up to and including today's lecture (Thursday, Oct 15th)

Topics Covered till date

Module 1: Propositional Logic

1. Syntax
2. Semantics
3. Formal proof systems
4. Normal Forms

Module 2: The Problem of Satisfiability

1. Propositional Resolution
2. DPLL
3. CDCL
4. SAT tool demo

Module 3: Predicate Logic

1. Syntax of predicate logic
2. Semantics of predicate logic
3. Natural deduction for FOL
4. Structures, power and limits of FOL
5. Normal forms, DPLL(T) and SMT

Module 4: Hoare Logic and Program Verification

1. Introduction to Hoare logic
2. Proof rules for partial correctness
3. Today: Loop invariants and variants

Recap: A proof calculus for program correctness

$$\frac{\{\varphi\}\ C_1\ \{\eta\} \qquad \{\eta\}\ C_2\ \{\psi\}}{\{\varphi\}\ C_1; C_2\ \{\psi\}}\ \text{Seq. Comp.} \qquad \frac{\{\varphi \wedge B\}\ C_1\ \{\psi\} \qquad \{\varphi \wedge \neg B\}\ C_2\ \{\psi\}}{\{\varphi\}\ \textbf{if}\ B\ \{C_1\}\ \textbf{else}\ \{C_2\}\ \{\psi\}}\ \text{If-then-Else}$$

$$\frac{}{\{\psi[E/x]\}\ x := E\ \{\psi\}}\ \text{Assignment} \qquad \frac{\vdash_{AR} \varphi \to \varphi' \qquad \{\varphi'\}\ C\ \{\psi'\} \qquad \vdash_{AR} \psi' \to \psi}{\{\varphi\}\ C\ \{\psi\}}\ \text{Implied}$$

$$\frac{\{\eta \wedge B\}\ C\ \{\eta\}}{\{\eta\}\ \textbf{while}\ B\ \{C\}\ \{\eta \wedge \neg B\}}\ \text{Partial-while}$$

We say sequent $\vdash_{par} \{\varphi\}\ P\ \{\psi\}$ is valid if ψ can be derived from φ by applying these rules!

This proof system is sound so we can use this to infer partial correctness, i.e., $\models_{par}$.

The dreaded 8am Pop quiz!

Consider program **Swap** defined as $t := x;\ x := y;\ y := t$, which is meant to swap x and y.

Look at the Hoare triple $\{x = x_0 \wedge y = y_0\}$ **Swap** $\{x = y_0 \wedge y = x_0\}$.

1. What are the pre and post conditions of this Hoare triple?
2. What are the program variables and what are the logical variables?
3. Show that this Hoare triple is valid, i.e., $\vdash_{par} \{x = x_0 \wedge y = y_0\}$ **Swap** $\{x = y_0 \wedge y = x_0\}$.

Solution

$\{x = x_0 \wedge y = y_0\}$

$\{y = y_0 \wedge x = x_0\}$ Implied

$t := x;$

$\{y = y_0 \wedge t = x_0\}$ Assignment

$x := y;$

$\{x = y_0 \wedge t = x_0\}$ Assignment

$y := t;$

$\{x = y_0 \wedge y = x_0\}$ Assignment

- x_0 and y_0 are logical variables: they remember the initial values (recall **Fac**$_2$)!

Exercise 21.1: Swap without a temporary

Show the same triple for the program $x := x + y;\ y := x - y;\ x := x - y$.

Topic 21.1

Proof Outlines and Loop Invariants

Recall: the If-then-Else rule and its variant

The rule

$$\frac{\{\varphi \wedge B\}\ C_1\ \{\psi\} \qquad \{\varphi \wedge \neg B\}\ C_2\ \{\psi\}}{\{\varphi\}\ \textbf{if}\ B\ \{C_1\}\ \textbf{else}\ \{C_2\}\ \{\psi\}}\ \text{If-then-Else}$$

The variant

$$\frac{\{\varphi_1\}\ C_1\ \{\psi\} \qquad \{\varphi_2\}\ C_2\ \{\psi\}}{\{(B \to \varphi_1) \wedge (\neg B \to \varphi_2)\}\ \textbf{if}\ B\ \{C_1\}\ \textbf{else}\ \{C_2\}\ \{\psi\}}\ \text{If-then-Else (variant)}$$

- **Quick question:** we know ψ and the two branches, but not φ. Which rule helps us *compute* a precondition?
- The variant: push ψ through C_1 and C_2 to get φ_1 and φ_2, then combine them as above.
- Together with Implied, the two rules are equivalent (**Exercise 20.5**).

Using the If-then-Else rule. E.g., $\vdash_{par} \{\top\}$ **Succ** $\{y = x + 1\}$ holds

$\{\top\}$	
$\{((x = 0) \to (0 = x)) \wedge (\neg(x = 0) \to (x + 1 = x + 1))\}$	Implied
$\{((x + 1 - 1 = 0) \to (1 = x + 1)) \wedge$	
$(\neg(x + 1 - 1 = 0) \to (x + 1 = x + 1))\}$	Implied
$a := x + 1;$	
$\{((a - 1 = 0) \to (1 = x + 1)) \wedge (\neg(a - 1 = 0) \to (a = x + 1))\}$	Assignment
if $(a - 1 = 0)\{$	
$\{1 = x + 1\}$	If-then-Else
$y := 1;$	
$\{y = x + 1\}$	Assignment
$\}$ **else** $\{$	
$\{a = x + 1\}$	If-then-Else
$y := a;$	
$\{y = x + 1\}$	Assignment
$\}$	
$\{y = x + 1\}$	If-then-Else

- **Quick question:** is $(B \to \varphi_1) \wedge (\neg B \to \varphi_2)$ the same as $(B \wedge \varphi_1) \vee (\neg B \wedge \varphi_2)$?
- Pushing ψ backwards like this is Dijkstra's weakest precondition calculus (1975): φ is $wp(\textbf{if } \ldots, \psi)$.



Edsger W. Dijkstra (1930–2002)

Are these still relevant 50 years later?



DOI:10.1145/3777544

Formal verification of AI workflows.

BY ERIK MEIJER

Guardians of the Agents

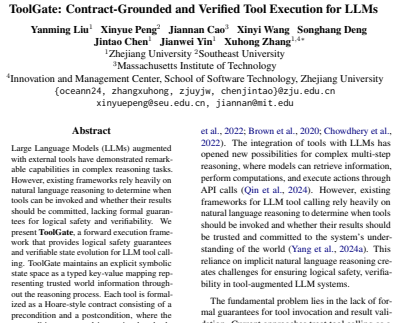
AGENTIC APPLICATIONS—AI SYSTEMS empowered to take autonomous actions by calling external tools—are the current rage in software development. They promise efficiency, convenience, and reduced human intervention. Giving autonomous agents access to

that these risks of ceding control to autonomous agents are sufficiently threatening that their use should be entirely prohibited.[a]

To mitigate the risks inherent in agentic applications, we propose a safety paradigm rooted in mathematical proof verification. In this design pattern, the AI agent is required to generate formal proofs demonstrating the safety of planned actions before being authorized to execute them. This approach parallels existing real-world practices (for example, credit card checksums, banknote security features), where complexity lies in production, but verification remains simple and efficient. Moreover, it extends the mechanism used by widely adopted computing platforms such as Java and .NET, where code undergoes a process of bytecode verification prior to execution.[b] Bytecode verification ensures crucial safety guarantees—notably memory safety, type correctness, and proper access control. Our proposal naturally extends this established principle, adapting it effectively to the emerging domain of agentic computations, thus providing robust assurances against harmful or unintended agent behavior.

The current state of the art in AI safety is to rely on evaluations (evals), which aim to identify potential risks, vulnerabilities, or harmful behaviors

E. Meijer, *Guardians of the Agents*, CACM 69(1), Jan. 2026



ToolGate: Contract-Grounded and Verified Tool Execution for LLMs

Yanming Liu[1] Xinyue Peng[2] Jiannan Cao[3] Xinyi Wang Songhang Deng
Jintao Chen[1] Jianwei Yin[1] Xuhong Zhang[1,4*]
[1]Zhejiang University [2]Southeast University
[3]Massachusetts Institute of Technology
[4]Innovation and Management Center, School of Software Technology, Zhejiang University
{oceann24, zhangxuhong, zjuyjw, chenjintao}@zju.edu.cn
xinyuepeng@seu.edu.cn, jiannan@mit.edu

Abstract

Large Language Models (LLMs) augmented with external tools have demonstrated remarkable capabilities in complex reasoning tasks. However, existing frameworks rely heavily on natural language reasoning to determine when tools can be invoked and whether their results should be committed, lacking formal guarantees for logical safety and verifiability. We present **ToolGate**, a forward execution framework that provides logical safety guarantees and verifiable state evolution for LLM tool calling. ToolGate maintains an explicit symbolic state space as a typed key-value mapping representing trusted world information throughout the reasoning process. Each tool is formalized as a Hoare-style contract consisting of a precondition and a postcondition, where the

et al., 2022; Brown et al., 2020; Chowdhery et al., 2022). The integration of tools with LLMs has opened new possibilities for complex multi-step reasoning, where models can retrieve information, perform computations, and execute actions through API calls (Qin et al., 2024). However, existing frameworks for LLM tool calling rely heavily on natural language reasoning to determine when tools should be invoked and whether their results should be trusted and committed to the system's understanding of the world (Yang et al., 2024a). This reliance on implicit natural language reasoning creates challenges for ensuring logical safety, verifiability in tool-augmented LLM systems.

The fundamental problem lies in the lack of formal guarantees for tool invocation and result vali-

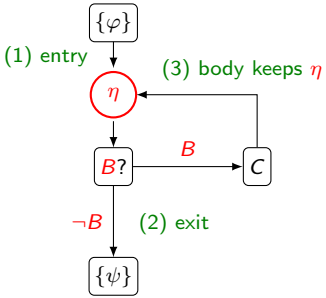
Liu et al., *ToolGate*, Findings of ACL 2026

> verification process relies on preconditions, postconditions, and invariants, ensuring only mathematically safe workflows proceed to execution.

CACM 2026, p. 51

Tackling the partial while!

- An invariant η holds before and after every iteration, even if not during! It is an induction hypothesis on the number of iterations: (1) is the base case, (3) the inductive step.
- **Exercise 21.2.** Give an example of a trivial invariant for any loop! Why does that not suffice?



The factorial example revisited

Fac$_1$ *inp* x *out* y	$y := 1;$ $z := 0;$ **while** $(z \neq x)\{$ $\quad z := z + 1;$ $\quad y := y * z;$ $\}$

- What is the invariant here?
- **Exercise 21.3**
  - Starting with $x = 6$, write the values of x, y, z before the loop and after each iteration.
- Notice that $y = z!$ holds every time the guard is tested!
- But for this to be our loop inv candidate we should check 2 other things.

- (1) Pre-condn of while loop implies it: $\vdash_{AR} (y = 1 \wedge z = 0) \rightarrow (y = z!)$ is valid.
- (2) Together with neg of guard, implies post-condn: $\vdash_{AR} (y = z! \wedge x = z) \rightarrow (y = x!)$ is valid.

So, let's write the full proof for the factorial program now!

The factorial example solved!

Fac$_1$ *inp* x *out* y	$y := 1;$ $z := 0;$ **while** $(z \neq x)\{$ $\quad z := z + 1;$ $\quad y := y * z;$ $\}$

$$\frac{\{\eta \wedge B\}\ C\ \{\eta\}}{\{\eta\}\ \textbf{while}\ B\ \{C\}\ \{\eta \wedge \neg B\}}$$

Partial-while applied, with $\eta := y = z!$ and $B := z \neq x$

(1) entry, (2) exit, (3) body keeps η

	$\{\top\}$	
	$\{1 = 0!\}$	Implied
	$y := 1;$	
	$\{y = 0!\}$	Assignment
	$z := 0;$	
(1)	$\{y = z!\}$	Assignment
	while $(z \neq x)\{$	
(3)	$\{y = z! \wedge z \neq x\}$	Inv Hyp. and Guard
	$\{y \cdot (z + 1) = (z + 1)!\}$	Implied
	$\quad z := z + 1$	
	$\{y \cdot z = z!\}$	Assignment
	$\quad y := y * z$	
	$\{y = z!\}$	Assignment
	$\}$	
(2)	$\{y = z! \wedge \neg(z \neq x)\}$	Partial-while
	$\{y = x!\}$	Implied

Another Factorial Example

Fac$_2$ *inp* x *out* y	$y := 1;$ **while** $(x \neq 0)\{$ $\quad y := y * x;$ $\quad x := x - 1;$ $\}$

Exercise 21.4 Consider program **Fac**$_2$

1. Does the same invariant (as for **Fac**$_1$) work for this? Why or why not?
2. Find a correct invariant (hint: use x_0) and show that
   $\vdash_{par} \{x = x_0 \wedge x \geq 0\}$ **Fac**$_2$ $\{y = x_0!\}$

In practice, use SMT Solvers to automate the checks.

- Proof steps like these can be handed to an SMT solver such as Z3 (a wrong invariant, e.g. $y = x!$, then gets a counterexample). Which theory is needed?
- Can it *synthesize* invariants? Not always! You (or an LLM?) have to provide it.
- Optional demo exercise with Z3 in next problem sheet!

Topic 21.2

Total Correctness

Total correctness

- **Quick question:** what happens if we run $\mathbf{Fac}_1$ with $x = -1$?
- The invariant $y = z!$ keeps holding, but z never reaches x: the loop *never exits*! So $\{\top\}\ \mathbf{Fac}_1\ \{y = x!\}$ is only partially correct.
- The partial correctness rules developed till now do not say anything if a program P loops indefinitely.
- Can we extend the proof calculus to show that programs also terminate?
- The only rule that causes looping is the while condn, so the only thing to change is the while rule!

Total correctness: the core idea

Core idea!

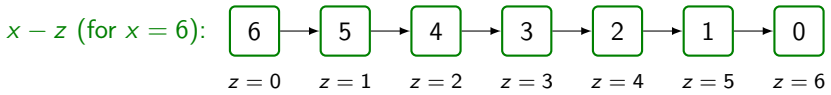
- for partial correctness of while loop, we have an invariant.
- for termination of while loop, we need something that varies but only a bounded no. of times.
  - In fact, we look for an integer expression that decreases every loop iteration, but always remains non-negative.
  - hence, it can decrease only finitely many times and hence the loop terminates.
  - Such an expression is called a variant.



Revisiting the Factorial example

	$y := 1;$
	$z := 0;$
Fac$_1$	**while** $(z \neq x)\{$
inp x	$\quad z := z + 1;$
out y	$\quad y := y * z;$
	$\}$

- What is the variant here?
- Can you find an integer expression whose value decreases at each loop iteration?
- easy! the expression $x - z$.
- Note that it is always nonnegative (as long as $x \geq 0$ initially).
- When it is 0, the loop terminates!



Proof rule for total correctness of while loops

For while loops

Total correctness = Partial correctness + Variant

$$\frac{\{\eta \wedge B \wedge 0 \leq E = E_0\}\ C\ \{\eta \wedge 0 \leq E < E_0\}}{\{\eta \wedge 0 \leq E\}\ \textbf{while}\ B\ \{C\}\ \{\eta \wedge \neg B\}}\ \text{Total-while}$$

- Variant E is the integer expression
  - whose value is equal to the logical variable E_0 before the loop body is executed
  - and after one execution of the loop body, the value is strictly less than E_0 but still non-negative.
- η is the invariant as before
- For $\textbf{Fac}_1$: $E = x - z$